

TECNOLOGÍAS DE LA INFORMACIÓN

Minería de Datos

3 créditos

Profesor:



Ricardo Ordoñez Avila, Mg.

Titulaciones	Semestre
MINERÍA DE DATOS	Sexto

Tutorías: El profesor asignado se publicará en el entorno virtual de aprendizaje online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección Cronograma de Actividades

PERÍODO ABRIL 2026 - AGOSTO 2026



Índice

Tabla de contenido

Tema 1: Minería de texto	4
1.1 Etapas del proceso	6
1.2 Análisis de texto y redes sociales.	13
Tema 2: Minería de datos espaciales	41
2.1 Definiciones	41
2.2 Construcción de aplicaciones geoespaciales	49



✓ Organización de la lectura para el estudiante por semana del compendio

Semanas	Compendio
Semana 13	Páginas 3 – 12
Semana 14	Páginas 13 – 36
Semana 15	Páginas 37 – 43
Semana 16	Páginas 44 – 52

Resultado de aprendizaje de la Unidad IV

Integrar soluciones de Tecnologías de la Información y las Comunicaciones y procesos empresariales para satisfacer las necesidades de información de las organizaciones, permitiéndoles llegar a sus objetivos de forma efectiva.

Unidad IV

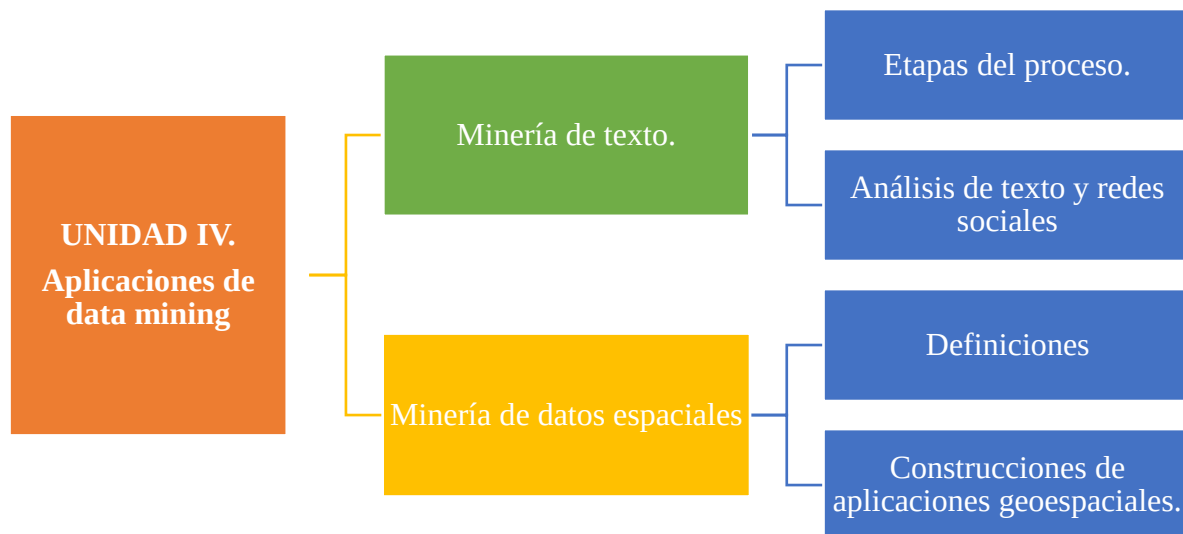
Aplicaciones de data mining

En el marco de la sociedad del conocimiento, en la que la gestión de la información, la documentación y el conocimiento se perfila como un componente estratégico de primer orden. Se han abierto los horizontes hacia otras profesiones que están llamadas a cooperar con los profesionales de la información: diseñadores de sistemas, proveedores de datos, editores, vendedores, archivistas, ingenieros y especialistas en codificación de texto electrónico, cuyas opiniones y experiencias permitan desarrollar las interfaces correspondientes para facilitar la localización, manipulación, recuperación y uso de la información digital.

Se han creado nuevas herramientas para facilitar el acceso al cúmulo de información que se genera diariamente. Una de las más utilizadas a nivel organizacional es la minería de texto (Text Mining) que ofrece a la organización la posibilidad de explorar grandes cantidades de textos, no organizados en forma de datos, establecer patrones y extraer conocimientos útiles (Botta-Ferret & Cabrera-Gato, 2007).

En este mismo orden, la propagación del uso de dispositivos de computación y de comunicación para la producción de información digital, y en particular en la producción de documentos textuales, ha generado la necesidad de desarrollar métodos, algoritmos y sistemas capaces de realizar el procesamiento automatizado de datos textuales estructurados, semi-estructurados y no estructurados para su organización y consulta, y con ello el surgimiento de áreas de estudio de la información como la minería de texto (Contreras, 2014).

Ejes Temáticos



Tema 1: Minería de texto

La minería de texto ha despertado un enorme interés en la comunidad científica, ya que ha aumentado su auge enormemente en los últimos años debido a la creciente cantidad de documentos disponibles en formato digital, y también a la creciente necesidad de organizar y obtener el conocimiento contenido en textos. La literatura en el campo de la minería de textos ofrece una serie amplia de aplicaciones tales como la clasificación supervisada, la recuperación de información, la clasificación no supervisada, extracción de entidades con nombre, encontrar tendencias mediante nubes de palabras, extraer resúmenes de grandes volúmenes de texto, La mayoría de las técnicas propuestas en este ámbito se basan en el paradigma del aprendizaje artificial. Un pilar importante de la minería de textos es la representación de documentos no estructurados de tal forma que reflejen los distintos rasgos de su contenido de la mejor manera posible (Guerrero, 2020).

La minería de texto es una aplicación de la minería de datos y consiste en descubrir o hallar, a partir de cantidades de información no estructurada el conocimiento del cual no existe ningún registro escrito. La información no estructurada es aquella que no está contenida en un "almacén" (base de datos), de forma

organizada para luego ser encontrada y utilizada fácilmente para distintos propósitos, lo que dificulta su extracción. Esta información puede estar representada en textos como los mensajes de correo electrónico, presentaciones en power point, documentos en word, mensajes instantáneos (Twitter, WhatsApp), software de colaboración (conferencias de video, salas de chat), entre otros; o se encuentra de forma no textual en imágenes de formato JPEG, archivos de audio MP3, correo de voz, etc (Guerrero, 2020).

En el marco empresarial, las organizaciones modernas disponen de muchos datos sobre sus clientes o su sector de actividad. Las nuevas tecnologías de lo digital como las redes sociales, el comercio electrónico o las aplicaciones móviles dan acceso a un gran volumen de información. Al analizar esos datos, es posible descubrir oportunidades no explotadas o problemas alarmantes que hay que resolver urgentemente. Sin embargo, algunos tipos de datos son más difíciles de explotar que otros (DataScientest, 2022).

Los datos procedentes de las redes sociales u otros sitios web son principalmente textos: comentarios sobre las publicaciones, reseñas sobre un producto, quejas en foros comunitarios, etc. No obstante, los textos forman parte de los datos llamados “no estructurados”. Esa información no puede ser tratada correctamente por softwares o herramientas de análisis de datos tradicionales. Por tanto, es necesario recurrir al “Text Mining” (DataScientest, 2022).

El Text Mining, o análisis de textos, consiste en transformar un texto no estructurado en datos estructurados para proceder posteriormente al análisis. Esa práctica se basa en la tecnología de “Natural Language Processing” (procesamiento natural del lenguaje), que permite que las máquinas comprendan y traten el lenguaje humano de manera automática (DataScientest, 2022).

La inteligencia artificial ahora es capaz de clasificar automáticamente los textos por sentimiento, tema o intención. Por ejemplo, un algoritmo de Text Mining es capaz de pasar revista a los comentarios sobre un producto para determinar si son principalmente positivos, neutros o negativos. También es posible localizar las palabras clave utilizadas con más frecuencia. De este modo, las empresas pueden analizar series de datos grandes y complejas con facilidad, rapidez y eficacia. Esta

disciplina también permite reducir el tiempo perdido en tareas manuales y repetitivas. Los equipos ahorran tiempo, y pueden centrarse en misiones más importantes que exigen una intervención humana. Los directivos de la empresa, por su parte, pueden apoyarse en los datos para tomar mejores decisiones (DataScientest, 2022).

En torno a sus aplicaciones, la minería de textos está presente en diferentes áreas; por ejemplo, en el ámbito de la medicina, la biología, la gestión documental o el análisis de opiniones, etc (UMA, 2022). En síntesis, la minería de datos se podrá aplicar en las áreas de:

- Extracción de información
- Análisis de sentimientos o minería de opiniones
- Clasificación documental
- Elaboración de resúmenes

1.1 Etapas del proceso

La minería de textos se encarga de descubrir información que no existía previamente de forma explícita, pero que surge de relacionar el contenido de varios de ellos. En UMA (2022) se mencionan tres procesos principales:

- Recuperación: selección de los textos oportunos.
- Extracción: extracción de la información incluida en esos textos mediante el procesamiento del lenguaje natural: acontecimientos, datos clave, hechos concretos, relaciones entre ellos, etc.
- Minería de datos: encontrar asociaciones entre los datos clave previamente extraídos de entre los textos.

Dichas actividades están enmarcadas dentro de tres fases:

1. *Fase de pre-procesamiento*: los textos se transforman en algún tipo de representación estructurada o semi-estructurada que facilite su análisis posterior. El primer paso dentro de la minería de texto sería definir el conjunto de documentos; además se debe evitar la duplicación de documentos dentro del corpus. Con el conjunto de documentos seleccionado y estructurado, debemos reconocer los tokens (unidades

gramaticales esenciales), lo que implica representar el texto como una lista de palabras mediante una representación vectorial.

2. *Etapas de descubrimiento:* las representaciones internas se analizan para descubrir patrones interesantes o nueva información.
3. *Etapas de visualización:* los usuarios pueden observar y explorar los resultados de una forma amena y sencilla.

De igual manera, Botta-Ferret & Cabrera-Gato (2007), mencionan tres etapas como técnicas clásicas del proceso de minería de texto: el pre-procesamiento, la representación y el descubrimiento. Respecto al pre-procesamiento, lo presentan como el proceso mediante el cual los textos se transforman en algún tipo de representación estructurada que facilite su análisis.

Como se puede observar, todas las etapas están muy interrelacionadas, así entonces, la primera etapa condiciona el descubrimiento de los patrones que la minería de texto puede realizar. La representación depende de la técnica de pre-procesamiento utilizada y determinará el algoritmo de descubrimiento a utilizar. Y el descubrimiento, por su parte, son algoritmos que, a partir de una representación estructurada de la información, son capaces de descubrir regularidades en los textos.



Figura 1. Proceso de minería de texto (Chang et al., 2018).

En relación a la figura 1, Chang et al. (2018), mencionan los siguientes pasos generales en la minería de texto:

- **Recolección:** Recopilación de datos de diferentes recursos, tales como sitio web, correos electrónicos, comentarios de clientes, archivo de documentos. Dependiendo de la aplicación, este proceso puede ser completamente automatizado o guiado por una persona encargada de realizar este proceso.
- **Preprocesamiento:** La identificación del contenido y la extracción de características representativas.
- **Limpieza de textos:** eliminación de cualquier información innecesaria o no deseada, como los anuncios de las páginas.
- **Tokenización:** un ordenador sólo «ve» una cadena de caracteres, sin poder identificar, por ejemplo, párrafos, frases o palabras. La Tokenización divide el texto en entidades significativas (palabras, oraciones, etc.) dados los espacios en blanco presentes y las puntuaciones.
- **Extracción de características (también llamada selección de atributos):** es el proceso de caracterización.

Por su parte, en DataScientest (2022), se mencionan algunos métodos y técnicas principales de la minería de texto, centrados en: las técnicas de análisis, la recuperación de la información, la clasificación de los textos, y la extracción de la información.

Las técnicas de análisis

La técnica de la “frecuencia de palabras” consiste en identificar los términos o conceptos más recurrentes en un conjunto de datos. Esto puede resultar muy útil, en particular para analizar las opiniones de clientes o las conversaciones en las redes sociales. Por ejemplo, si términos como “demasiado caro” o “sobrevalorado” aparecen con frecuencia, el análisis puede sugerir que el producto es demasiado caro. Por tanto, es necesario ajustar el precio en la medida de lo posible.

El método de colocación, por su parte, consiste en localizar las secuencias de palabras que aparecen con frecuencia las unas cerca de las otras. Algunas palabras

aparecen muy a menudo juntas. Se puede tratar de bigramas o trigramas, combinaciones de dos o tres palabras. Al identificar esas colocaciones, es posible comprender mejor la estructura semántica de un texto y obtener resultados de Text Mining más fiables.

El método de la concordancia se suele utilizar para reconocer el contexto en el que aparece un conjunto de palabras en un texto. Esta técnica permite evitar la ambigüedad y comprender el sentido de un término en el contexto específico.

La recuperación de información

La recuperación de información consiste en encontrar información pertinente a partir de un conjunto predefinido de consultas o frases. A menudo se emplea este enfoque en los sistemas de catálogos de bibliotecas o los motores de búsqueda web. Los sistemas “IR” (information retrieval) utilizan diferentes algoritmos para seguir los comportamientos de los usuarios e identificar los datos pertinentes.

La “**tokenización**” consiste en descomponer un texto largo en frases o palabras llamadas “tokens”. Esos tokens se usan después en los modelos para el clustering de texto o las tareas que buscan asociar documentos. El “**stemming**” consiste en separar los prefijos y los sufijos de las palabras para derivar su raíz y su significado. Esta técnica permite reducir el tamaño de los archivos de indexación.

La clasificación de textos

También existen métodos más avanzados de Text Mining. La clasificación de textos consiste en asignar etiquetas a los datos de texto no estructurados. Es una fase esencial e indispensable para el tratamiento natural del lenguaje (Natural Language Processing). Efectivamente, permite organizar y estructurar un texto complejo para despejar los datos pertinentes. Gracias a esta técnica, las empresas están capacitadas para analizar todo tipo de información textual para sacar valiosas orientaciones.

Existen diferentes formas de clasificación de texto. El análisis de temas (Topic Analysis) permite comprender los temas principales de un texto. Es una de las razones principales para organizar los datos textuales. El análisis de sentimientos

consiste en analizar las emociones contenidas en un texto. Esto permite delimitar mejor las opiniones de la clientela, por ejemplo, revisando los comentarios sobre un producto. Los textos pueden clasificarse según si son positivos, negativos o neutros. La detección del lenguaje consiste en clasificar un texto en función del lenguaje. Por ejemplo, será posible clasificar las consultas a un servicio de atención al cliente para redirigirlas hacia un asesor o un agente que domine el lenguaje adecuado. Esto permite ganar un tiempo muy valioso. Por último, la detección de intenciones permite reconocer de manera automática las intenciones de un texto. Por ejemplo, el análisis de diferentes respuestas a un correo electrónico publicitario permite determinar qué interlocutores están interesados en un producto.

La extracción de información

Otra técnica del Text Mining es la extracción de texto. Tiene el objetivo de extraer datos específicos en un texto, como palabras clave, nombres propios, direcciones o correos electrónicos. Esto permite evitar tener que clasificar los datos manualmente y, por tanto, se gana tiempo.

En resumen, la minería de textos cuenta con una metodología propia para llevar a cabo el proceso de extracción de información. Según Guerrero (2020), dicha metodología puede ser entendida como el proceso mediante el cual se llevan a cabo una serie de tareas ordenadas orientadas para la consecución de tres objetivos principales:

1. Establecimiento del corpus (conjunto de textos).
2. Transformación de documentos no estructurados a estructurados.
3. Extracción del conocimiento.

A continuación, se amplían algunos conceptos empleados en la minería de texto, según Guerrero (2020):

Bag of Words (Bolsa de palabras, BoW): El modelo bag-of-words (bolsa de palabras, BoW) en el procesamiento de lenguaje natural es un método popular para representar documentos, que ignora el orden de las palabras. El modelo BoW permite un modelado basado en diccionario, y cada documento parece una bolsa (de ese modo no se considera el orden), que contiene algunas palabras del diccionario, cada

palabra constituye una posición de un vector y el valor corresponde con el número de veces que ha aparecido. Para complementar el proceso de bolsa de palabras existen técnicas para reducir el número de palabras en el conjunto de documentos, tales como stemming o raíz, lexeme o lematización, eliminar palabras muertas.

Stop words (Palabras muertas): son palabras frecuentes dentro de una cadena de texto. Generalmente, son palabras comunes tales como el, la, y, pero, etc. En inglés, sería: the, and, but, etc. Estas palabras ayudan a formar ideas, pero no suelen tener valor individualmente.

Lematización: relaciona una palabra flexionada o derivada con su forma canónica o lema. Y un lema no es otra cosa que la forma que tienen las palabras cuando las buscas en el diccionario. por ejemplo, canto, cantas, canta, cantamos, cantáis, cantan son distintas formas (conjugaciones) de un mismo verbo (cantar). Y que niña, niño, niñita, niños, niñotes, y otras más, son distintas formas del vocablo niño. Así que la lematización encuentra estas diferencias y agrupa todas estas variantes en un mismo término.

La lematización es un proceso clave en muchas tareas prácticas de procesamiento de lenguaje natural (PLN), pero tiene dos costos. Primero, es un proceso que consume recursos (sobre todo tiempo). Segundo, suele ser probabilística, así que en algunos casos obtendremos resultados inesperados.

Stemming (Raíz): es el proceso de reducir las palabras de una cadena de texto a su raíz. Estas raíces son la parte invariable de palabras relacionadas sobre todo por su forma. Por ejemplo, las palabras intelligence e intelligent se reducen a la raíz intellig. El stemming es mucho más rápido desde el punto de vista del procesamiento que la lematización. También tiene como ventaja que reconoce relaciones entre palabras de distinta clase. Podría reconocer, por ejemplo, que picante y picar tienen como raíz pic-. En otras palabras, el stemming puede reducir el número de elementos que forman nuestros textos. Y eso, en muchos casos, es lo que buscamos. Como desventaja pueden “recortar” demasiado la raíz y encontrar relaciones entre palabras que realmente no existen, un stemmer sólo procesa una palabra y desconoce el contexto en donde se ubica dicha palabra, suele ser una buena solución cuando no importa demasiado la precisión y se requiere de un procesamiento eficiente.

Term Frequency — Inverse Document Frequency (Término de Frecuencia - Frecuencia inversa del documento, TF-IDF): esta es una técnica para cuantificar una palabra en documentos, generalmente calculamos un peso para cada palabra que significa la importancia de la palabra en el documento y el corpus. Este método es una técnica ampliamente utilizada en recuperación de información y minería de texto. Aumenta a medida que aumenta el número de apariciones de esa palabra en el documento, se equilibra por el peso o frecuencia del termino en la colección de documentos, permitiendo manejar el hecho de que algunas palabras son más comunes que otras, es decir incorpora un factor de frecuencia inversa de documento que atenúa el peso de los términos que ocurren con mucha frecuencia en la colección de documentos e incrementa el peso de los términos que ocurren pocas veces.

$TF-IDF = \text{Frecuencia de término (TF)} * \text{Frecuencia de documento inversa (IDF)}$

t - término (palabra)

d - documento (conjunto de palabras)

N - recuento de corpus

corpus - el conjunto total de documentos

Term Frequency (Término de Frecuencia, TF): Mide la frecuencia de una palabra en un documento. Esto depende en gran medida de la longitud del documento y de la generalidad de la palabra, por ejemplo, una palabra muy común como "fue" puede aparecer varias veces en un documento. pero si se toma dos documentos, uno con 100 palabras y otro con 10000 palabras. Hay una alta probabilidad de que la palabra común como "fue" pueda estar presente más en el documento redactado de 10000. Pero no se puede afirmar que el documento más largo es más importante que el documento más corto. Por esta razón exacta, TF normaliza el valor de frecuencia, dividiendo la frecuencia con el número total de palabras en el documento.

$tf(t,d) = \text{número de veces que aparece } t \text{ en } d / \text{número de palabras en } d$

Cuando estamos vectorizando los documentos, verificamos el recuento de cada palabra. En el peor de los casos, si el término no existe en el documento, ese valor de TF particular será 0 y, en otro caso extremo, si todas las palabras en el documento son iguales, entonces será 1. El valor final de la normalización El valor de TF estará en el rango de [0 a 1]. 0, 1 inclusive.

Document Frequency, Frecuencia de Documento, DF: Mide la importancia del documento en todo el conjunto de corpus, esto es muy similar a TF. La única diferencia es que TF es un contador de frecuencia para un término t en el documento d , DF es el recuento de ocurrencias del término t en el conjunto de documentos N . En otras palabras, DF es el número de documentos en los que la palabra está presente. Consideramos una ocurrencia si el término consiste en el documento al menos una vez, no necesitamos saber la cantidad de veces que el término está presente.

$df(t)$ = aparición de t en documentos

Topic Modeling Latent Dirichlet Allocation (Modelado de temas, LDA): El modelado de tópicos es una técnica que suele aplicarse para el agrupamiento y clasificación de datos en una colección de textos.

1.2 Análisis de texto y redes sociales.

Twitter es actualmente una dinámica e ingente fuente de contenidos que, dada su popularidad e impacto, se ha convertido en la principal fuente de información para estudios de Social Media Analytics. Análisis de reputación de empresas, productos o personalidades, estudios de impacto relacionados con marketing, extracción de opiniones y predicción de tendencias son sólo algunos ejemplos de aplicaciones (Amat, 2017).

A continuación se presenta el caso práctico desarrollado por Amat (2017), en el que se realiza la automatización de la extracción de datos de Twitter, se analiza las palabras empleadas por cada uno de los usuarios, se realiza experimentos básicos de la autoría, y el análisis de sentimientos

Algunos paquetes usados en este ejercicio son:

- **stringr**: paquete desarrollado por Hadley Wickham con multitud de funciones (división, búsqueda, reemplazo...) para trabajar con strings.
- **tidytext**: paquete desarrollado por Julia Silge y David Robinson. Los autores proponen una forma de trabajar con texto que sigue los principios de tidy data, lo que hace muy sencillo combinarlo con otros paquetes tales como dplyr, broom, tidyr y ggplot2.
- **quanteda**: paquete con multitud de funciones orientadas a text mining, algunas de ellas permiten crear Term-Document Matrices
- **purrr**: permite aplicar funciones a elementos de un vector o lista, por ejemplo, a los elementos de una columna de un dataframe.

1.2.1 Extracción de datos Twitter.

La plataforma Twitter, pone a disposición de los usuarios, una API, donde se podrá extraer información. Para poder extraer datos de Twitter es necesario estar registrado en la plataforma y, a partir de la cuenta, crear una Twitter App asociada. Al crear una Twitter App, Twitter proporciona una serie de claves y tokens de identificación que permiten acceder a la aplicación y extraer información.

Se puede crear una APP en <https://developer.twitter.com/en> y de cómo crear y acceder a ella a través de R en <https://cran.r-project.org/web/packages/rtweet/vignettes/auth.html> Twitter tiene una normativa que regula la frecuencia máxima de peticiones, así como la cantidad máxima de tweets que se pueden extraer rate limiting. Es importante cumplir estos límites para evitar ser bloqueado.

En el ejercicio propuesto por Amat (2017), se analizan los tweets de: Elon Musk (@elonmusk) y Bill Gates (@BillGates), dos directivos de empresas tecnológicas, y de Mayor Ed Lee (@mayoredlee) alcalde de la ciudad de San Francisco. Para el trabajo de análisis que se quiere realizar, es conveniente recuperar tantos tweets como sea posible, o al menos unos 1000. Como el número máximo de tweets recuperados por consulta es de 200 y existe una limitación de tiempo mínimo entre consulta y consulta, se sigue la siguiente estrategia:

- Todo tweet tiene un ID global numérico que sigue un orden temporal, lo que permite identificar si un tweet es más reciente que otro.
- Entre los argumentos de `api.GetUserTimeline()` se puede especificar el `max_id` para recuperar únicamente tweets más antiguos.
- Antes de cada consulta, se lee el fichero donde se están almacenando los tweets y se identifica el ID del último tweet recuperado. Si no existe fichero de almacenamiento para el usuario en cuestión, se crea uno.
- Se realiza una nueva consulta empleando como argumento `max_id` el ID recuperado en el paso anterior.
- Se incorpora los nuevos datos al archivo de almacenamiento.

```
library(rtweet)
library(tidyverse)
library(knitr)

# Identificación y obtención de tokens
appname <- "extrac-----eets"
key <- "WXkVypH-----mY0"
secret <- "EeMD2-----qYbYw5EFTKR7i3M"
twitter_token <- create_token(app = appname, consumer_key = key,
                              consumer_secret = secret)

extraccion_tweets <- function(usuario, maxtweets = 100, output_file_name = NULL){
  # Esta función extrae los tweets publicados por un usuario y los almacena en
  # un fichero csv. Si existe un fichero con el mismo nombre, lo lee, concatena
  # los nuevos tweets y lo sobrescribe.
  #
  # Argumentos:
  #   usuario: identificador del usuario de twitter
  #   maxtweets: número de tweets que se recuperan
  #   output_file_name: nombre del fichero de escritura

  # Si no se especifica el nombre del archivo de almacenamiento, se crea un
  # nombre por defecto
  if(is.null(output_file_name)){
    output_file_name <- paste0("datos_tweets_", usuario, ".csv")
  }

  # Si no existe el fichero de almacenamiento, se crea uno nuevo con los
  # resultados de la primera recuperación
  if(!(output_file_name %in% list.files())){
    datos_new <- get_timeline(user = usuario, n = maxtweets, parse = TRUE,
                              check = TRUE, include_rts = FALSE)
    write_csv(x = datos_new, path = output_file_name, col_names = TRUE)
    print("Nuevo fichero creado")
  }else{
    # Se leen los datos antiguos
    datos_old <- read_csv(file = output_file_name)
    # Se identifica el último Id recuperado
    ultimo_id <- tail(datos_old, 1)["status_id"] %>% pull()
```

```
# Para no recuperar de nuevo el último tweet de la consulta anterior
# se incrementa en 1 el Id
ultimo_id = ultimo_id + 1
# Para que no haya errores de compatibilidad, se convierten todas las
# columnas numéricas a character
datos_old <- map_if(.x = datos_old, .p = is.numeric, .f = as.character)
# Extracción de nuevos tweets
datos_new <- get_timeline(user = usuario, n = maxtweets, max_id = ultimo_id,
                          parse = TRUE, check = TRUE, include_rts = FALSE)
datos_new <- map_if(.x = datos_new, .p = is.numeric, .f = as.character)
# Concatenación de los datos nuevos, viejos y escritura en disco
datos_concatenados <- bind_rows(datos_old, datos_new)
write_csv(x = datos_concatenados, path = output_file_name, col_names = TRUE)
print(paste("Número total de tweets:", nrow(datos_concatenados)))
print(paste("Número de tweets nuevos:", nrow(datos_new)))
}
```

Cada vez que se ejecuta la función `extraccion_tweets()` se recuperan nuevos tweets y se almacenan junto a los extraídos previamente.

```
extraccion_tweets(usuario = "@elonmusk", maxtweets = 200)
extraccion_tweets(usuario = "@BillGates", maxtweets = 200)
extraccion_tweets(usuario = "@mayoredlee", maxtweets = 200)
```

Esta sección puede ser utilizada una vez, que se gestione la obtención de datos desde la API de Twitter, siguiendo la estrategia de reunir los datos mediante el uso de la función propuesta.

1.2.2 Carga de datos.

Para el proceso de carga de datos, desde el aula virtual de la asignatura, se podrá descargar directamente los datasets en archivo delimitado por comas, (`datos_tweets_@BillGates`, `datos_tweets_@elonmusk`, y `datos_tweets_@mayoredlee`) disponibles en el aula virtual.

```
tweets_elon <- read_csv(file = "./datos/datos_tweets_@elonmusk.csv",
                        col_names = TRUE)
tweets_BillGates <- read_csv(file = "./datos/datos_tweets_@BillGates.csv",
                             col_names = TRUE)
```



```
tweets_mayoredlee <- read_csv(file = "../datos/datos_tweets_@mayoredlee.csv",  
                               col_names = TRUE)
```

```
# Se unen todos los tweets en un único dataframe
```

```
tweets <- bind_rows(tweets_elon, tweets_BillGates, tweets_mayoredlee)  
tweets %>% group_by(screen_name) %>% summarise(numero_tweets = n())
```

```
## # A tibble: 3 x 2  
##   screen_name numero_tweets  
##   <chr>          <int>  
## 1 BillGates      2087  
## 2 elonmusk       2678  
## 3 mayoredlee     2447
```

El número de tweets recuperados es superior a 2000 para todos los usuarios. De entre toda la información disponible, en este análisis únicamente se emplea: autor del tweet, fecha de publicación, identificador del tweet y contenido.

```
colnames(tweets)
```

```
## [1] "screen_name"          "user_id"  
## [3] "created_at"           "status_id"  
## [5] "text"                 "retweet_count"  
## [7] "favorite_count"       "is_quote_status"  
## [9] "quote_status_id"      "is_retweet"  
## [11] "retweet_status_id"    "in_reply_to_status_status_id"  
## [13] "in_reply_to_status_user_id" "in_reply_to_status_screen_name"  
## [15] "lang"                 "source"  
## [17] "media_id"             "media_url"  
## [19] "media_url_expanded"   "urls"  
## [21] "urls_display"         "urls_expanded"  
## [23] "mentions_screen_name" "mentions_user_id"  
## [25] "symbols"              "hashtags"  
## [27] "coordinates"          "place_id"  
## [29] "place_type"           "place_name"  
## [31] "place_full_name"      "country_code"  
## [33] "country"              "bounding_box_coordinates"  
## [35] "bounding_box_type"
```

```
# Selección de variables
tweets <- tweets %>% select(screen_name, created_at, status_id, text)

# Se renombran las variables con nombres más prácticos
tweets <- tweets %>% rename(autor = screen_name, fecha = created_at,
                             texto = text, tweet_id = status_id)

head(tweets)
```

```
## # A tibble: 6 x 4
##   autor      fecha      tweet_id texto
##   <chr>      <dtm>      <dbl> <chr>
## 1 elonmusk 2017-11-09 17:28:57 9.29e17 "\"If one day, my words are again...
## 2 elonmusk 2017-11-09 17:12:46 9.29e17 "I placed the flowers\n\nThree br...
## 3 elonmusk 2017-11-08 18:55:13 9.28e17 Atatürk Anıtkabir https://t.co/al...
## 4 elonmusk 2017-11-07 19:48:45 9.28e17 @Bob_Richards One rocket, slightl...
## 5 elonmusk 2017-10-28 21:36:18 9.24e17 @uncover007 500 ft so far. Should...
## 6 elonmusk 2017-10-28 21:30:41 9.24e17 Picture of The Boring Company LA ...
```

1.2.3 Limpieza de texto y Tokenización.

El proceso de limpieza de texto, dentro del ámbito de text mining, consiste en eliminar del texto todo aquello que no aporte información sobre su temática, estructura o contenido. No existe una única forma de hacerlo, depende en gran medida de la finalidad del análisis y de la fuente de la que proceda el texto. Por ejemplo, en las redes sociales los usuarios pueden escribir de la forma que quieran, lo que suele resultar en un uso elevado de abreviaturas y signos de puntuación. En este ejercicio, dado que los principales objetivos son estudiar el perfil lingüístico de los tres usuarios, identificar la autoría de los tweets y analizar el sentimiento que transmiten, se procede a eliminar:

- Patrones no informativos (urls de páginas web)
- Signos de puntuación

- Etiquetas HTML
- Caracteres sueltos
- Números

Tokenizar un texto consiste en dividir el texto en las unidades que lo conforman, entendiendo por unidad el elemento más sencillo con significado propio para el análisis en cuestión, en este caso, las palabras.

Existen múltiples librerías que automatizan en gran medida la limpieza y tokenización de texto, por ejemplo, tokenizers o quanteda. Sin embargo, creo que se entiende mejor el proceso implemento una función propia que, si bien puede estar menos optimizada, es más transparente. Definir una función que contenga cada uno de los pasos de limpieza tiene la ventaja de poder adaptarse fácilmente dependiendo del tipo de texto analizado.

```
limpiar_tokenizar <- function(texto){  
  # El orden de la limpieza no es arbitrario  
  # Se convierte todo el texto a minúsculas  
  nuevo_texto <- tolower(texto)  
  # Eliminación de páginas web (palabras que empiezan por "http." seguidas  
  # de cualquier cosa que no sea un espacio)  
  nuevo_texto <- str_replace_all(nuevo_texto, "http\\S*", "")  
  # Eliminación de signos de puntuación  
  nuevo_texto <- str_replace_all(nuevo_texto, "[[:punct:]]", " ")  
  # Eliminación de números  
  nuevo_texto <- str_replace_all(nuevo_texto, "[[:digit:]]", " ")  
  # Eliminación de espacios en blanco múltiples  
  nuevo_texto <- str_replace_all(nuevo_texto, "[\\s]+", " ")  
  # Tokenización por palabras individuales  
  nuevo_texto <- str_split(nuevo_texto, " ")[[1]]  
  # Eliminación de tokens con una longitud < 2  
  nuevo_texto <- keep(.x = nuevo_texto, .p = function(x){str_length(x) > 1})  
  return(nuevo_texto)  
}  
  
test = "Esto es 1 ejemplo de 1'limpieza de6 TEXTO https://t.co/rnHPgyhx4Z  
@JoaquinAmatRodrigo #textmining"  
limpiar_tokenizar(texto = test)
```

```
## [1] "esto"          "es"              "ejemplo"
## [4] "de"            "limpieza"        "de"
## [7] "texto"         "joaquinamatrodrigo" "textmining"
```

Puede observarse que la función `limpiar_tokenizar()` elimina el símbolo @ y # de las palabra a las que acompañan. En Twitter, los usuarios se identifican de esta forma, por lo que @ y # pertenecen al nombre.

Aunque es importante tener en cuenta las eliminaciones del proceso de limpieza, el impacto en este caso no es demasiado alto, ya que, si un documento se caracteriza por tener la palabra #datascience, también será detectado fácilmente mediante la palabra datascience.

```
# Se aplica la función de limpieza y tokenización a cada tweet
tweets <- tweets %>% mutate(texto_tokenizado = map(.x = texto,
                                                    .f = limpiar_tokenizar))
tweets %>% select(texto_tokenizado) %>% head()
```

```
## # A tibble: 6 x 1
##   texto_tokenizado
##   <list>
## 1 <chr [13]>
## 2 <chr [15]>
## 3 <chr [2]>
## 4 <chr [6]>
## 5 <chr [18]>
## 6 <chr [9]>
```

Gracias a la característica de las tibble de poder contener cualquier tipo de elemento en sus columnas (siempre que sea el mismo para toda la columna), se puede almacenar el texto tokenizado. Cada elemento de la columna `texto_tokenizado` es una lista con un vector de tipo character que contiene los tokens generados.

```
tweets %>% slice(1) %>% select(texto_tokenizado) %>% pull()
```

```
## [[1]]
## [1] "if"      "one"      "day"      "my"      "words"    "are"      "against"
## [8] "science" "choose"   "science"  "mustafa" "kemal"    "atatürk"
```

1.2.4 Análisis exploratorio.

En R, las estructuras por excelencia para el análisis exploratorio son el `DataFrame` y la `Tibble`, que es la forma en la que se encuentra almacenada ahora la información de los tweets. Sin embargo, al realizar la tokenización, ha habido un cambio importante. Previamente a la división del texto, los elementos de estudio (observaciones) eran los tweets, y cada uno se encontraba en una fila, cumplimento así la condición de tidy data: una observación, una fila.

Al realizar la tokenización, el elemento de estudio ha pasado a ser cada token (palabra), incumpliendo así la condición de tidy data. Para volver de nuevo a la estructura ideal se tiene que expandir cada lista de tokens, duplicando el valor de las otras columnas tantas veces como sea necesario. Ha este proceso se le conoce como expansión o `unnest`.

```
tweets_tidy <- tweets %>% select(-texto) %>% unnest()
tweets_tidy <- tweets_tidy %>% rename(token = texto_tokenizado)
head(tweets_tidy)
```

```
## # A tibble: 6 x 4
##   autor      fecha                tweet_id token
##   <chr>      <dtm>                  <dbl> <chr>
## 1 elonmusk 2017-11-09 17:28:57  9.29e17 if
## 2 elonmusk 2017-11-09 17:28:57  9.29e17 one
## 3 elonmusk 2017-11-09 17:28:57  9.29e17 day
## 4 elonmusk 2017-11-09 17:28:57  9.29e17 my
## 5 elonmusk 2017-11-09 17:28:57  9.29e17 words
## 6 elonmusk 2017-11-09 17:28:57  9.29e17 are
```

Ahora que la información está en formato *tidy*, se pueden realizar filtrados, sumatorios y representaciones con gran facilidad. La función `unnest_tokens()` del paquete `tidytext` permite, entre otras cosas, automatizar el proceso tokenización y almacenamiento en formato *tidy* en un único paso.

1.2.5 Distribución temporal de los tweets

Dado que cada usuario puede haber iniciado su actividad en Twitter en diferente momento, es interesante explorar si los tweets recuperados solapan en el tiempo.

```
library(lubridate)

ggplot(tweets, aes(x = as.Date(fecha), fill = autor)) +
  geom_histogram(position = "identity", bins = 20, show.legend = FALSE) +
  scale_x_date(date_labels = "%m-%Y", date_breaks = "5 month") +
  labs(x = "fecha de publicación", y = "número de tweets") +
  facet_wrap(~ autor, ncol = 1) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90))
```

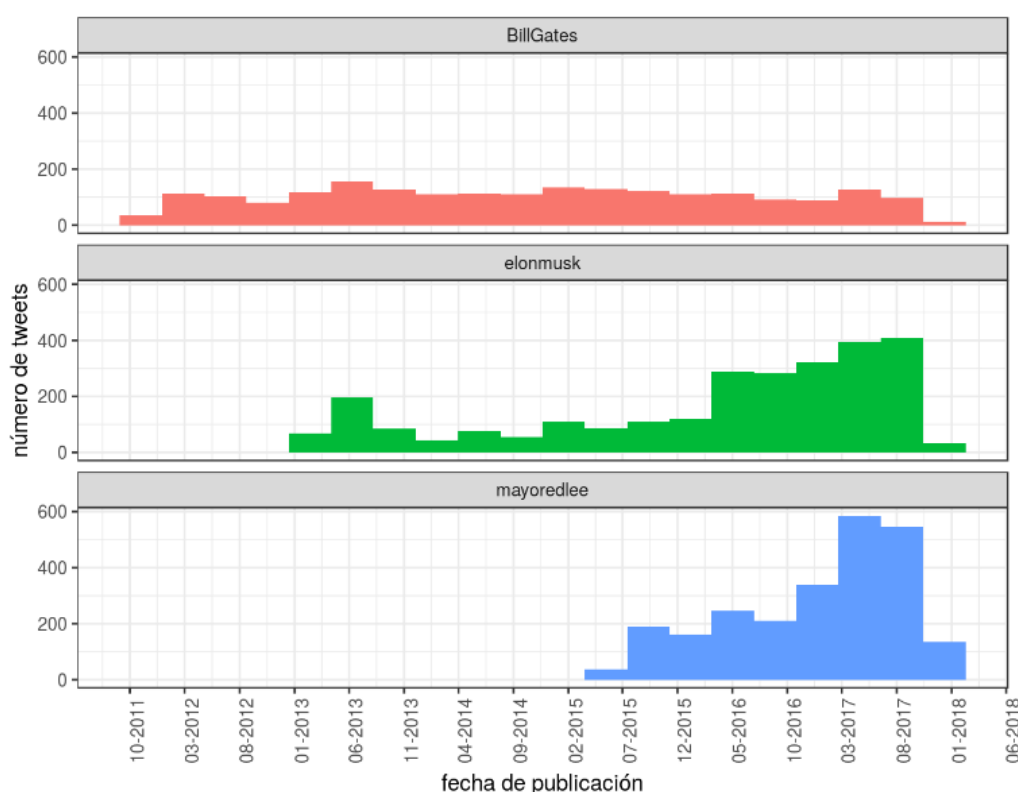


Figura 2. Frecuencia de los tweets en el tiempo, por usuario (Amat, 2017).

```

tweets_mes_ano <- tweets %>% mutate(mes_ano = format(fecha, "%Y-%m"))
tweets_mes_ano %>% group_by(autor, mes_ano) %>% summarise(n = n()) %>%
  ggplot(aes(x = mes_ano, y = n, color = autor)) +
  geom_line(aes(group = autor)) +
  labs(title = "Número de tweets publicados", x = "fecha de publicación",
        y = "número de tweets") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, size = 6),
        legend.position = "bottom")
    
```

En la figura 3, se podrá observar un perfil de actividad distinto para cada usuario. Bill Gates ha mantenido una actividad constante de en torno a 30 tweets por mes durante todo el periodo estudiado. Elon Musk muestra una actividad inicial por debajo de la de Bill Gates pero, a partir de febrero de 2016 incrementó sustancialmente el número de tweets publicados. Ed Lee tiene una actividad muy alta sobre todo en el periodo 2017.

Debido a las limitaciones que impone Twitter en las recuperaciones, cuanto más activo es un usuario, menor es el intervalo de tiempo para el que se recuperan tweets. En el caso de Ed Lee, dado que publica con mucha más frecuencia que el resto, con la misma cantidad de tweets recuperados se abarca menos de la mitad del rango temporal que con los otros.

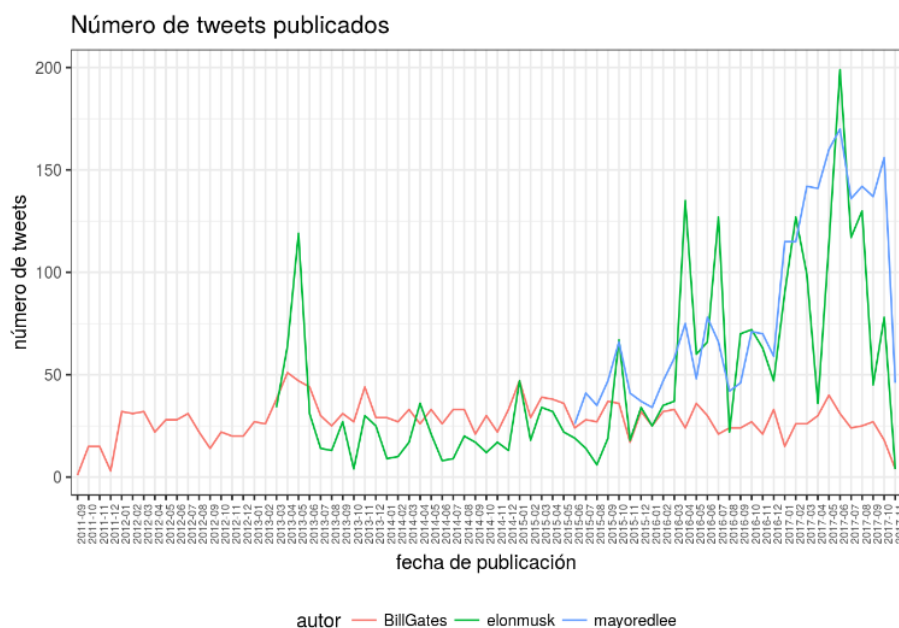


Figura 3. Frecuencia de los tweets en 30 días, por usuario (Amat, 2017).

1.2.6 Frecuencia de palabras

A la hora de entender que caracteriza la escritura de cada usuario, es interesante estudiar qué palabras emplea, con qué frecuencia, así como el significado de las mismas.

1.2.6.1 Total, de palabras utilizadas por cada usuario:

```
tweets_tidy %>% group_by(autor) %>% summarise(n = n())
```

```
## # A tibble: 3 x 2  
##   autor      n  
##   <chr>    <int>  
## 1 BillGates 31572  
## 2 elonmusk  33584  
## 3 mayoredlee 41787
```

```
tweets_tidy %>% ggplot(aes(x = autor)) + geom_bar() + coord_flip() + theme_bw()
```

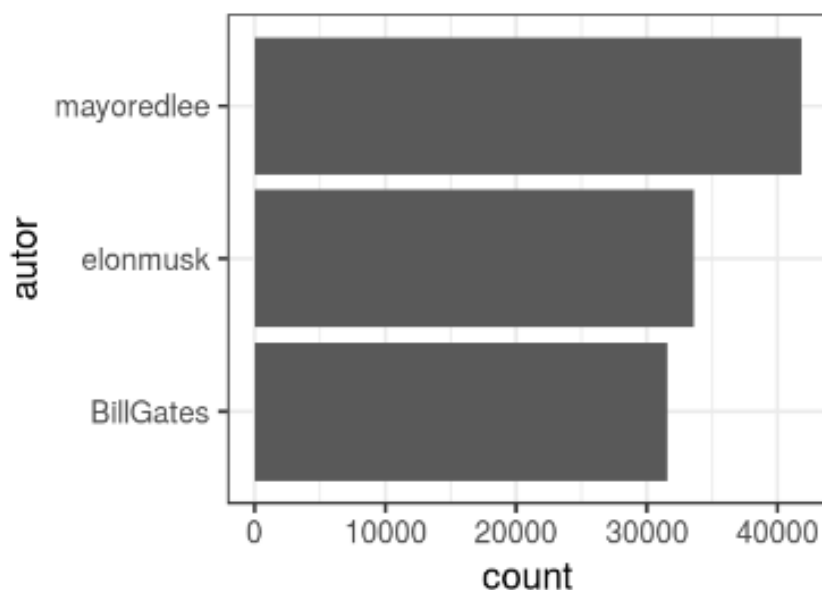


Figura 3. Total, de palabras, por usuario (Amat, 2017).

1.2.6.2 Palabras distintas utilizadas por cada usuario

```
tweets_tidy %>% select(autor, token) %>% distinct() %>% group_by(autor) %>%
  summarise(palabras_distintas = n())
```

```
## # A tibble: 3 x 2
##   autor      palabras_distintas
##   <chr>          <int>
## 1 BillGates      4510
## 2 elonmusk       6552
## 3 mayoredlee     5471
```

```
tweets_tidy %>% select(autor, token) %>% distinct() %>%
  ggplot(aes(x = autor)) + geom_bar() + coord_flip() + theme_bw()
```

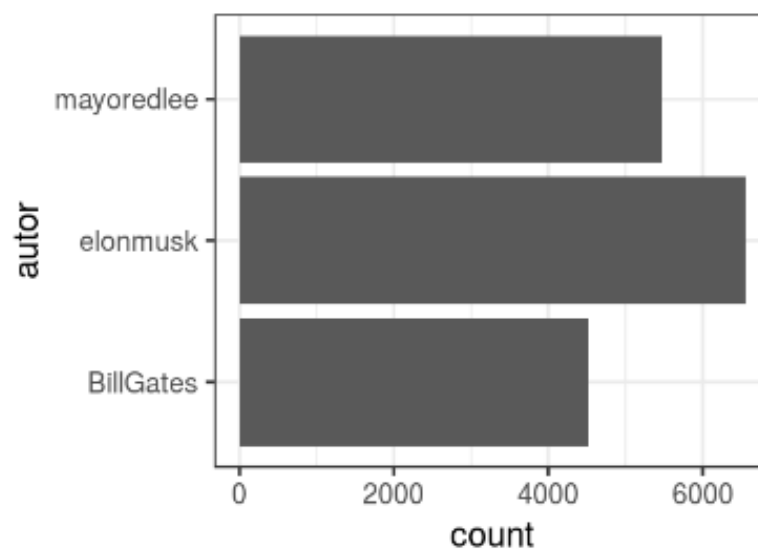


Figura 4. Total de palabras distintas, por usuario (Amat, 2017).

Aunque Elon Musk no es el que más palabras totales ha utilizado, bien porque ha publicado menos tweets o porque estos son más cortos, es el que más palabras distintas emplea.

1.2.6.3 Longitud media de los tweets por usuario

```

tweets_tidy %>% group_by(autor, tweet_id) %>% summarise(longitud = n()) %>%
group_by(autor) %>% summarise(media_longitud = mean(longitud),
                                sd_longitud = sd(longitud))
    
```

```

## # A tibble: 3 x 3
##   autor      media_longitud sd_longitud
##   <chr>          <dbl>         <dbl>
## 1 BillGates      15.2           3.35
## 2 elonmusk       12.6           6.90
## 3 mayoredlee     17.1           3.45
    
```

```

tweets_tidy %>% group_by(autor, tweet_id) %>% summarise(longitud = n()) %>%
group_by(autor) %>%
  summarise(media_longitud = mean(longitud),
            sd_longitud = sd(longitud)) %>%
  ggplot(aes(x = autor, y = media_longitud)) +
  geom_col() +
  geom_errorbar(aes(ymin = media_longitud - sd_longitud,
                    ymax = media_longitud + sd_longitud)) +
  coord_flip() + theme_bw()
    
```

En la figura 5, se presenta la longitud de los tweets. El tipo de tweet de Bill Gates y Mayor Ed Lee es similar en cuanto a longitud media y desviación. Elon Musk alterna más entre tweets cortos y largos, siendo su media inferior a la de los otros dos.

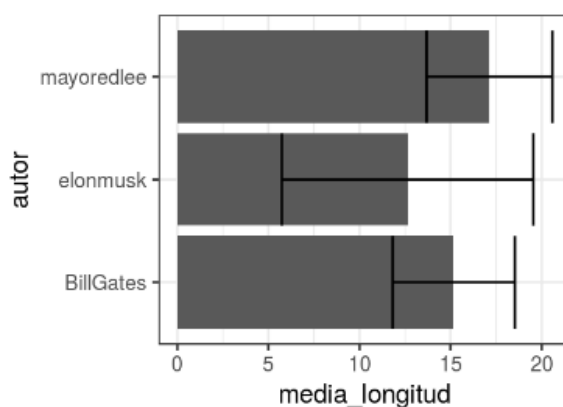


Figura 5. Longitud de los tweets, por usuario (Amat, 2017).

1.2.6.4 Palabras más utilizadas por usuario

```
tweets_tidy %>% group_by(autor, token) %>% count(token) %>% group_by(autor) %>%  
  top_n(10, n) %>% arrange(autor, desc(n)) %>% print(n=30)
```

```
## # A tibble: 30 x 3  
## # Groups:   autor [3]  
##   autor      token      n  
##   <chr>      <chr> <int>  
## 1 BillGates the      1195  
## 2 BillGates to        1117  
## 3 BillGates of         670  
## 4 BillGates in         591  
## 5 BillGates is         453  
## 6 BillGates and         439  
## 7 BillGates for         363  
## 8 BillGates this        345  
## 9 BillGates we          334  
## 10 BillGates on          333  
## 11 elonmusk the          988  
## 12 elonmusk to           916  
## 13 elonmusk of           638  
## 14 elonmusk is           543  
## 15 elonmusk in           478  
## 16 elonmusk for          400  
## 17 elonmusk and          367  
## 18 elonmusk it           351  
## 19 elonmusk on           344  
## 20 elonmusk that         315  
## 21 mayoredlee to        1693  
## 22 mayoredlee the        1355  
## 23 mayoredlee amp        1212  
## 24 mayoredlee our        1104  
## 25 mayoredlee sf          944  
## 26 mayoredlee for         822  
## 27 mayoredlee of          819  
## 28 mayoredlee we          782  
## 29 mayoredlee in          780  
## 30 mayoredlee is          452
```

1.2.6.5 Stop words

En el resultado anterior se puede observar que los términos más frecuentes en todos los usuarios se corresponden con artículos, preposiciones, pronombres..., en general, palabras que no aportan información relevante sobre el texto. Ha estas palabras se les conoce como stopwords. Para cada idioma existen distintos listados de stopwords, además, dependiendo del contexto, puede ser necesario adaptar el listado. En la tabla anterior aparece el término amp que procede de la etiqueta html &. Con frecuencia, a medida que se realiza un análisis se encuentran palabras que deben incluirse en el listado de stopwords. Dentro del paquete tidytext y tokenizers pueden encontrarse varios listados de stopwords para los idiomas “en”, “da”, “de”, “el”, “es”, “fr”, “it”, “ru”. En este caso, se emplea un listado de stopwords obtenido de la librería de Python nltk.corpus.

```
lista_stopwords <- c('me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves',  
                    'you', 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his',  
                    'himself', 'she', 'her', 'hers', 'herself', 'it', 'its', 'itself',  
                    'they', 'them', 'their', 'theirs', 'themselves', 'what', 'which',  
                    'who', 'whom', 'this', 'that', 'these', 'those', 'am', 'is', 'are',  
                    'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had',  
                    'having', 'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and',  
                    'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at',  
                    'by', 'for', 'with', 'about', 'against', 'between', 'into',  
                    'through', 'during', 'before', 'after', 'above', 'below', 'to',  
                    'from', 'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under',  
                    'again', 'further', 'then', 'once', 'here', 'there', 'when',  
                    'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few', 'more',  
                    'most', 'other', 'some', 'such', 'no', 'nor', 'not', 'only', 'own',  
                    'same', 'so', 'than', 'too', 'very', 's', 't', 'can', 'will',  
                    'just', 'don', 'should', 'now', 'd', 'll', 'm', 'o', 're', 've',  
                    'y', 'ain', 'aren', 'couldn', 'didn', 'doesn', 'hadn', 'hasn',  
                    'haven', 'isn', 'ma', 'mightn', 'mustn', 'needn', 'shan',  
                    'shouldn', 'wasn', 'weren', 'won', 'wouldn', 'i')  
  
# Se añade el término amp al listado de stopwords  
lista_stopwords <- c(lista_stopwords, "amp")  
  
# Se filtran las stopwords  
tweets_tidy <- tweets_tidy %>% filter(!(token %in% lista_stopwords))
```

1.2.6.6 Representación gráfica de las frecuencias

```
tweets_tidy %>% group_by(autor, token) %>% count(token) %>% group_by(autor) %>%
  top_n(10, n) %>% arrange(autor, desc(n)) %>%
  ggplot(aes(x = reorder(token, n), y = n, fill = autor)) +
  geom_col() +
  theme_bw() +
  labs(y = "", x = "") +
  theme(legend.position = "none") +
  coord_flip() +
  facet_wrap(~autor, scales = "free", ncol = 1, drop = TRUE)
```

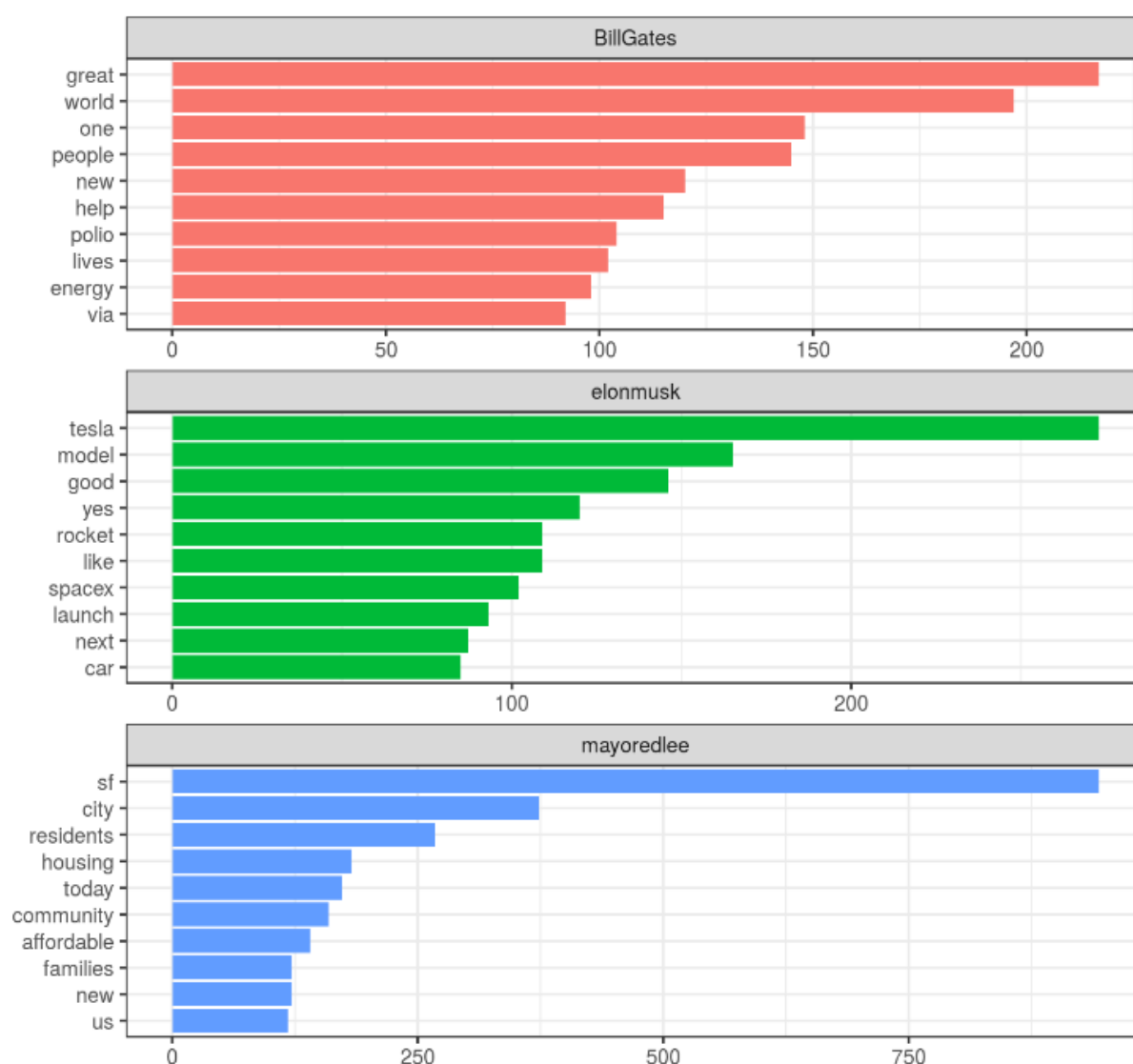


Figura 6. Palabras frecuentes, por usuario (Amat, 2017).

Los resultados obtenidos tienen sentido si ponemos en contexto la actividad profesional de los usuarios analizados. Mayor Ed Lee es alcalde de San Francisco (sf), por lo que sus tweets están relacionados con la ciudad, residentes, familias, casas... Elon Musk dirige varias empresas tecnológicas entre las que destacan Tesla y SpaceX, dedicadas a los coches y a la aeronáutica. Por último, Bill Gates, además de propietario de microsoft, dedica parte de su capital a fundaciones de ayuda, de ahí las palabras mundo, polio, ayuda...

1.2.6.7 Word Clouds

Otra forma visual de representar las palabras más frecuentes es mediante nubes de palabras (*word clouds*). En esta representación, las palabras más importantes tienen mayor tamaño.

```
library(wordcloud)
library(RColorBrewer)

wordcloud_custom <- function(grupo, df){
  print(grupo)
  wordcloud(words = df$token, freq = df$frecuencia,
            max.words = 400, random.order = FALSE, rot.per = 0.35,
            colors = brewer.pal(8, "Dark2"))
}

df_grouped <- tweets_tidy %>% group_by(autor, token) %>% count(token) %>%
  group_by(autor) %>% mutate(frecuencia = n / n()) %>%
  arrange(autor, desc(frecuencia)) %>% nest()

walk2(.x = df_grouped$autor, .y = df_grouped$data, .f = wordcloud_custom)
```

```
## [1] "BillGates"
```



Figura 7. Nubes de palabras, usuario BillGates (Amat, 2017).

```
## [1] "elonmusk"
```



Figura 8. Nubes de palabras, usuario elonmusk (Amat, 2017).

[1] "mayoredlee"



Figura 9. Nubes de palabras, usuario mayoredlee (Amat, 2017).

1.2.7 Análisis de sentimientos

Una forma de analizar el sentimiento de un de un texto es considerando su sentimiento como la suma de los sentimientos de cada una de las palabras que lo forman. Esta no es la única forma abordar el análisis de sentimientos, pero consigue un buen equilibrio entre complejidad y resultados. Para llevar a cabo esta aproximación es necesario disponer de un diccionario en el que se asocie a cada palabra un sentimiento o nivel de sentimiento. A estos diccionarios también se les conoce como sentiment lexicon. El paquete tidytext contiene 3 diccionarios distintos:

- AFINN: asigna a cada palabra un valor entre -5 y 5. Siendo -5 el máximo de negatividad y +5 el máximo de positividad.
- Bing: clasifica las palabras de forma binaria positivo/negativo.
- nrc: clasifica cada palabra en uno o más de los siguientes sentimientos: positive, negative, anger, anticipation, disgust, fear, joy, sadness, surprise, and trust.

En este ejercicio se emplea la clasificación positivo/negativo proporcionada por el diccionario bing.

```
library(tidytext)
sentimientos <- get_sentiments(lexicon = "bing")
head(sentimientos)
```

```
## # A tibble: 6 x 2
##   word      sentiment
##   <chr>     <chr>
## 1 2-faced   negative
## 2 2-faces   negative
## 3 a+       positive
## 4 abnormal negative
## 5 abolish  negative
## 6 abominable negative
```

Para facilitar el cálculo de sentimientos globales (autor, tweet...) se recodifican los sentimientos como +1 para positivo y -1 para negativo.

```
sentimientos <- sentimientos %>%
  mutate(valor = if_else(sentiment == "negative", -1, 1))
```

1.2.7.1 Sentimiento promedio en cada Tweet.

Al disponer de los datos en formato tidy (una palabra por fila), mediante un inner join se añade a cada palabra su sentimiento y se filtran automáticamente todas aquellas palabras para las que no hay información disponible.

```
tweets_sent <- inner_join(x = tweets_tidy, y = sentimientos,
  by = c("token" = "word"))
```

Se suman los sentimientos de las palabras que forman cada tweet.

```
tweets_sent %>% group_by(autor, tweet_id) %>%
  summarise(sentimiento_promedio = sum(valor)) %>%
  head()
```

```
## # A tibble: 6 x 3
## # Groups:   autor [1]
##   autor      tweet_id sentimiento_promedio
##   <chr>      <dbl>          <dbl>
## 1 BillGates  1.20e17             4
## 2 BillGates  1.21e17            -1
## 3 BillGates  1.22e17             2
## 4 BillGates  1.22e17             3
## 5 BillGates  1.27e17             1
## 6 BillGates  1.27e17             1
```

1.2.7.2 Porcentaje de tweets positivos, negativos y neutros por autor

```
tweets_sent %>% group_by(autor, tweet_id) %>%
  summarise(sentimiento_promedio = sum(valor)) %>%
  group_by(autor) %>%
  summarise(positivos = 100 * sum(sentimiento_promedio > 0) / n(),
            neutros = 100 * sum(sentimiento_promedio == 0) / n(),
            negativos = 100 * sum(sentimiento_promedio < 0) / n())
```

```
## # A tibble: 3 x 4
##   autor      positivos neutros negativos
##   <chr>      <dbl>    <dbl>    <dbl>
## 1 BillGates    72.4    11.3    16.4
## 2 elonmusk     63.6    11.3    25.1
## 3 mayoredlee    79.7     6.26   14.1
```

```
tweets_sent %>% group_by(autor, tweet_id) %>%
  summarise(sentimiento_promedio = sum(valor)) %>%
  group_by(autor) %>%
```

```

summarise(positivos = 100*sum(sentimiento_promedio > 0) / n(),
          neutros = 100*sum(sentimiento_promedio == 0) / n(),
          negativos = 100*sum(sentimiento_promedio < 0) / n()) %>%
ungroup() %>%
gather(key = "sentimiento", value = "valor", -autor) %>%
ggplot(aes(x = autor, y = valor, fill = sentimiento)) +
geom_col(position = "dodge", color = "black") + coord_flip() +
theme_bw()
    
```

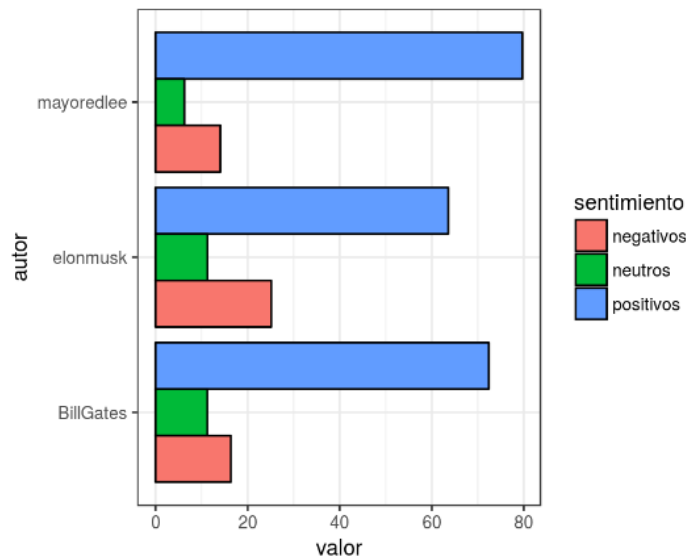


Figura 10. Porcentaje de tweets negativos, positivos y neutros por usuario (Amat, 2017).

Los tres autores tienen un perfil muy similar. La gran mayoría de tweets son de tipo positivo. Este patrón es común en redes sociales, donde se suele participar mostrando aspectos o actividades positivas. Los usuarios no tienden a mostrar las cosas malas de sus vidas.



Recuerde que: El paquete tidytext brinda acceso a varios léxicos de sentimientos. Tres léxicos de propósito general son: AFINN, Bing, y NRC (Silge & Robinson, 2016).

1.2.7.3 Evolución de los sentimientos en función del tiempo

A continuación, se estudia como varía el sentimiento promedio de los tweets agrupados por intervalos de un mes para cada uno de los usuarios.

```

library(lubridate)
tweets_sent %>% mutate(anyo = year(fecha),
  mes = month(fecha),
  anyo_mes = ymd(paste(anyo, mes, sep="-"), truncated=2)) %>%
  group_by(autor, anyo_mes) %>%
  summarise(sentimiento = mean(valor)) %>%
  ungroup() %>%
  ggplot(aes(x = anyo_mes, y = sentimiento, color = autor)) +
  geom_point() +
  geom_smooth() +
  labs(x = "fecha de publicación") +
  facet_wrap(~ autor, ncol = 1) +
  theme_bw() +
  theme(legend.position = "none")
  
```

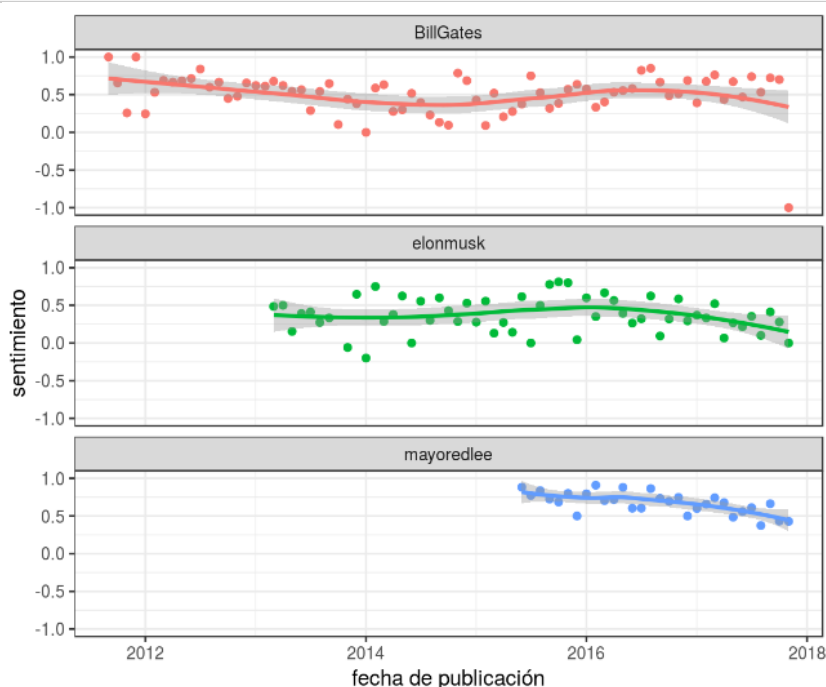


Figura 11. Evolución de los sentimientos en el tiempo, por usuario (Amat, 2017).

La distribución del sentimiento promedio de los tweets se mantiene aproximadamente constante para los 3 usuarios. Existen ciertas oscilaciones, pero todas ellas dentro del rango de sentimiento positivo.

1.2.8 Web scraping

Puede ocurrir que la plataforma de la que se desea extraer información no disponga de una API de consultas, o bien que imponga limitaciones en cuanto a la cantidad de información que se puede obtener (como es el caso de Twitter). En estas situaciones, se puede intentar extraer la información directamente de la web, lo que se conoce como web scraping. Las páginas web son en su mayoría archivos html que contienen toda la información en forma de texto y que el explorador (Chrome, Firefox, Safari...) interpreta para crear lo que vemos cuando navegamos por internet. El paquete rvest es una herramienta excelente para extraer información de archivos html.

```
library(tidyverse)
library(rvest)

# Scrapping de la web twitter
# =====
twitter_BillGate <- read_html("https://twitter.com/billgates?lang=es")
tweets <- twitter_BillGate %>% html_nodes(".js-tweet-text-container") %>%
  html_text()
length(tweets)
```

```
## [1] 20
```

A pesar de que, según la página, @BillGates tiene 2588 publicaciones, solo se han recuperado 20. Esto ocurre porque la página sigue una estrategia de carga de tipo infinite scroll, en lugar de cargar todo el contenido a la vez, se hace a medida que el usuario avanza en la página. ¿Cómo recuperarlos todos? Por suerte existe el paquete Rselenium que permite, entre otras muchas cosas, avanzar en la barra de navegación de una web de forma automática.

```
library(Rselenium)
driver <- rsDriver()
```

```
## [1] "Connecting to remote server"
## $acceptInsecureCerts
## [1] FALSE
##
## $acceptSslCerts
## [1] FALSE
##
## $applicationCacheEnabled
## [1] FALSE
##
## $browserConnectionEnabled
## [1] FALSE
##
## $browserName
## [1] "chrome"
##
## $chrome
## $chrome$chromedriverVersion
## [1] "2.40.565383 (76257d1ab79276b2d53ee976b2c3e3b9f335cde7) "
##
## $chrome$userDataDir
## [1] "/tmp/.org.chromium.Chromium.pB9UFe"
##
##
## $cssSelectorsEnabled
## [1] TRUE
##
## $databaseEnabled
## [1] FALSE
##
## $handlesAlerts
## [1] TRUE
##
## $hasTouchScreen
## [1] FALSE
##
## $javascriptEnabled
## [1] TRUE
##
## $locationContextEnabled
## [1] TRUE
##
## $mobileEmulationEnabled
## [1] FALSE
```

```
##
## $nativeEvents
## [1] TRUE
##
## $networkConnectionEnabled
## [1] FALSE
##
## $pageLoadStrategy
## [1] "normal"
##
## $platform
## [1] "Linux"
##
## $rotatable
## [1] FALSE
##
## $setWindowRect
## [1] TRUE
##
## $takesHeapSnapshot
## [1] TRUE
##
## $takesScreenshot
## [1] TRUE
##
## $unexpectedAlertBehaviour
## [1] ""
##
## $version
## [1] "67.0.3396.79"
##
## $webStorageEnabled
## [1] TRUE
##
## $webdriver.remote.sessionid
## [1] "97cc2d82cad3efa7ce6fe961cd275f35"
##
## $id
## [1] "97cc2d82cad3efa7ce6fe961cd275f35"
```

```
driver$client$navigate(url = "https://twitter.com/billgates?lang=es")
```

```
# Se ejecuta escape para salir del formulario de inicio de sesión
```

```
driver$client$sendKeysToActiveElement(sendKeys = list(key = "escape"))

# Se avanza en la página para que se cargue su contenido iteración a iteración
for(i in 1:20){
  driver$client$sendKeysToActiveElement(sendKeys = list(key = "end"))
  # Se deja un tiempo para que cargue
  Sys.sleep(0.5)
}

# Se recupera el contenido de la página
source_pagina <- driver$client$getPageSource()

# Se cierra la ventana y el servidor
driver$client$closeWindow()
driver$client$closeServer()
```

```
## {xml_document}
## <html lang="en">
## [1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset= ...
## [2] <body>\n\n<div id="content">\n  <div id="help-heading">\n    <h1><sp ...
```

```
# Se lee el contenido html con read_html()
twitter_BillGate <- read_html(source_pagina[[1]])
tweets <- twitter_BillGate %>% html_nodes(".js-tweet-text-container") %>%
  html_text()
length(tweets)
```

```
## [1] 200
```

En este ejemplo se ha avanzado solo 20 veces en la página, pero podría incrementarse para que se cargaran muchos más tweets (necesitaría bastante tiempo). Para agilizar el proceso, se puede configurar el explorador para que no cargue los vídeos e imágenes (Amat, 2017).

Tema 2: Minería de datos espaciales

Los datos espaciales, o más bien los datos geoespaciales, se refieren a los datos obtenidos de lugares geográficos, es decir, de zonas de la superficie de la Tierra. El uso de este tipo de datos reconoce “la importancia de conceptos espaciales como distancia, ubicación, proximidad, vecindad y región en la sociedad humana,” permitiéndonos enfrentar los fenómenos desde una perspectiva multivariante y multidimensional, proporcionándonos información adicional para nuestras observaciones (Escobar & Ortiz, 2021).

En ciencia política, la perspectiva espacial para el análisis de datos ha avanzado sobre cómo hacer inferencias ecológicas, y sobre la accesibilidad a los lugares de votación. Este tipo de datos permite responder a preguntas como la influencia de la proximidad de los países vecinos en la difusión de ciertas políticas, pero también resulta ser una herramienta clave en la exploración de datos para observar la existencia de patrones territoriales en otros estudios (Escobar & Ortiz, 2021).

En los últimos años, los gobiernos que proporcionan datos para diversos procesos sociales y económicos del país. Las nuevas plataformas de acceso abierto, como Google Maps u Open Street Map, y las redes sociales como Twitter y Facebook también contribuyeron al interés por los datos espaciales. Mientras tanto, para los investigadores y estudiantes, las bases de datos como la ya mencionada del Depósito de Datos Espaciales de la Universidad de Nueva York proporcionan archivos shapefile de fácil acceso y manipulación para generar visualizaciones y unir datos georreferenciados de otras bases de datos (Escobar & Ortiz, 2021).

2.1 Definiciones

En R, existen varias clases de objetos para representar información espacial. Según, Fernández & Cotos (2021), en el campo de los datos espaciales se suele distinguir entre dos tipos de datos:

- Datos **vectoriales**: en los que se emplean coordenadas para definir las posiciones espaciales “exactas” de los datos. Entre ellos estarían los asociados a las geometrías habituales: puntos, líneas, polígonos y rejillas.

- Datos **ráster**: se utilizan habitualmente para representar una superficie continua. Un ráster no es más que una rejilla regular que determina un conjunto de rectángulos denominados celdas (o píxeles en el análisis de imágenes de satélite y teledetección) que tienen asociados uno o más valores. Este tipo de datos también se denominan arrays o data cubes espaciales (o espaciotemporales).

2.1.1 Datos vectoriales: modelo simple feature.

El modelo geométrico de rasgo simple (simple feature) es un estándar de código abierto desarrollado y respaldado por el Open Geospatial Consortium (OGC) para representar una amplia gama de información geográfica. Es un modelo de datos jerárquico que simplifica los datos geográficos al condensar un amplio rango de formas geográficas en una única clase de geometría. Simple feature es un modelo de datos ampliamente respaldado que subyace a las estructuras de datos en muchas aplicaciones de SIG, incluidas QGIS y PostGIS, permitiendo la transferencia de datos a otras configuraciones (Mas, 2018).

El paquete **sf** es totalmente compatible con los tipos de simple feature utilizados en la gran mayoría de las operaciones de análisis espacial: puntos, líneas, polígonos y sus respectivas versiones "múltiples". El paquete **sf** permite también crear colecciones de geometrías, que pueden contener diferentes tipos de geometría en un solo objeto. **sf** incorpora de forma integral la funcionalidad de los tres paquetes principales del paradigma **sp** para el sistema de clases de objetos espaciales, **rgdal** para leer y escribir datos y **rgeos** para operaciones espaciales.

En R, los objetos simple feature se almacenan en una tabla dataframe, en la cual los datos geográficos ocupan una columna especial, que contiene una lista. Esta columna generalmente se llama "geom" o "geometry".

En **sf**, existen diferentes clases de objetos espaciales dependiendo del tipo de información. Las clases **Point** y **Multipoint**, **Linestring** y **Multilinestring** y las clases **Polygon** y **Multipolygon** permiten manejar respectivamente coberturas de puntos, líneas y polígonos. La clase **Geometrycollection** permite juntar, en un mismo objeto, diferentes geometrías (Mas, 2018).

2.1.1.1 Cobertura de puntos.

El paquete **sf** permite manejar datos en dos, tres o cuatro dimensiones. Típicamente, la tercera dimensión es la altura (z) y la cuarta (m) alguna medición (temperatura por ejemplo). Las coordenadas se presentan en forma de vector o de tabla y se transforman en una cobertura de puntos de la clase sfg: simple feature geometry con la función `st_point()` (un solo punto) o `st_multipoint()` (varios puntos) (Mas, 2018).

```

library(sf) # install.packages("sf") si necesario instalar sf
# Geometrías Simple Features (sfg)
# Point: coordenadas de un punto en 2, 3 o 4 dimensiones
P <- st_point(c(2,5)) # 2 dimensiones (XY)
class(P)

## [1] "XY"      "POINT" "sfg"

P <- st_point(c(2,5,17,44),"XYZM") # 4 dimensiones (XYZM)
class(P)

## [1] "XYZM"    "POINT" "sfg"

str(P)

## Classes 'XYZM', 'POINT', 'sfg'  num [1:4] 2 5 17 44

plot(P, axes = TRUE)

```

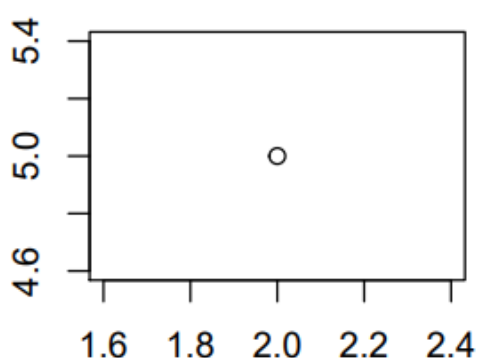


Figura 12. Cobertura de un solo punto (Mas, 2018).

```
# Multipoint: coordenadas de varios puntos en 2, 3 o 4 dimensiones
# Crea un vector of coordenadas en x
Xs <- c(2,4,5)
# Crea un vector of coordenadas en y
Ys <- c(5,4,8)
# Pega Xs y Ys para crear una tabla de coordenadas
coords <- cbind(Xs,Ys)
print(coords)

##          Xs Ys
## [1,]    2  5
## [2,]    4  4
## [3,]    5  8

# Crea el objeto Multipoint (MP)
MP <- st_multipoint(coords)
plot(MP, axes = TRUE)
```

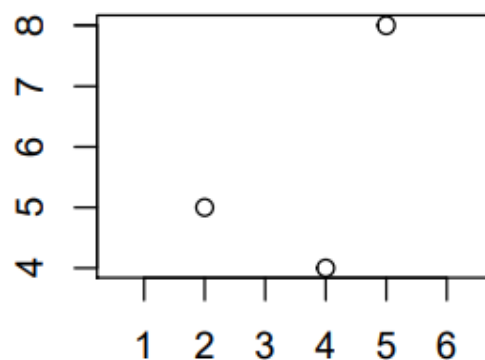


Figura 13. Cobertura de varios puntos (Mas, 2018).

2.1.1.2 Cobertura de líneas.

En formato vector, una línea simple está definida por las coordenadas de los vértices. Para el manejo de este tipo de datos, sf maneja objetos de la clase **Linestring** que describen segmentos simples (que no presentan intersecciones o bifurcaciones). Los objetos Multilinestring agrupan segmentos Linestring.

```
# Crea 3 objetos "Linestring": simple cadena de coordenadas (vértices)
# Línea L1
X1s <- c(0,3,5,8,10)
Y1s <- c(0,3,4,8,10)
Coord1 <- cbind(X1s,Y1s)
# Crea objeto de Clase Linestring
L1 <- st_linestring(Coord1)
class(L1)

## [1] "XY"          "LINESTRING" "sfg"

print(L1)

## LINESTRING (0 0, 3 3, 5 4, 8 8, 10 10)

# Línea L2
X2s <- c(2,1,1)
Y2s <- c(2,4,5)
Coord2 <- cbind(X2s,Y2s)
# Crea objeto de Clase Linestring
L2 <- st_linestring(Coord2)

# Línea 3
X3s <- c(8,8)
Y3s <- c(8,5)
Coord3 <- cbind(X3s,Y3s)
# Crea objeto de Clase Linestring
L3 <- st_linestring(Coord3)
```

En el código anterior, creamos tres objetos Linestring (L1, L2 y L3) con base en una tabla de coordenadas usando la función **st_linestring()**. De la misma forma que para los puntos, se pueden juntar objetos en colecciones (sfc) con la función **st_sfc()**

```
# Crea un objeto Multilineas: conjunto de objetos Linestring
L1L2 <- st_multilinestring(list(L1,L2))

# Junta varios sfg en un sfc (colección de simple features)
geometria2 <- st_sfc(L1,L2,L3)
```

```
# Tabla de atributos
num <- c(1,2,3)
code <- c("t","t","p")
tipo <- c("Terraceria","Terraceria","Pavimentada")
tablineas <- data.frame(cbind(num,tipo,code))
print(tablineas)

##   num      tipo code
## 1    1 Terraceria   t
## 2    2 Terraceria   t
## 3    3 Pavimentada   p

# sf object
SFL <- st_sf(tablineas, geometry = geometria2)
plot(SFL, axes=TRUE)
```

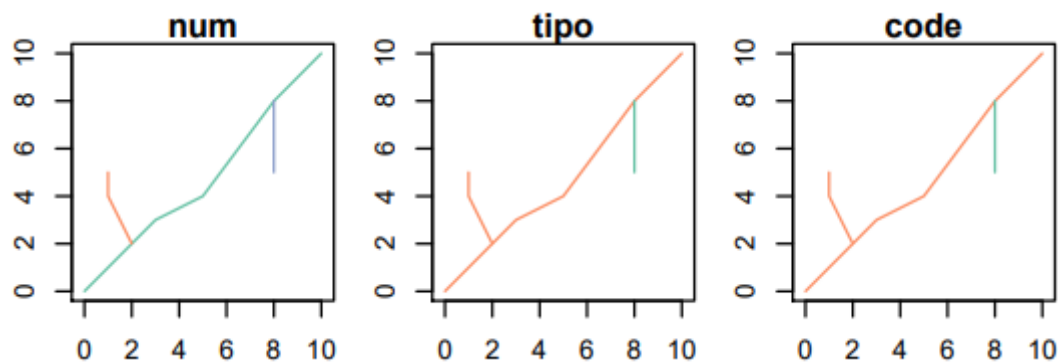


Figura 14. Cobertura de líneas (Mas, 2018).

2.1.1.2 Cobertura de polígonos.

Para crear una cobertura de polígonos, se crean objetos Polygon a partir de las coordenadas de vértices que crean una forma cerrada utilizando la función **st_polygon()**. La secuencia de los vértices es en sentido de las manecillas del reloj para delimitar “huecos” dentro de otro polígono y en sentido contrario para el contorno externo. Finalmente, se agrupan varios objetos en una colección utilizando la función **st_sfc()**.

```
## P1 Polígono forestal al SudEste
## Polygon
# Crea una cadena de coordenadas en X
X1 <- c(5,10,10,6,5)
# Crea una cadena de coordenadas en Y
# Ojo tiene que cerrar (último par de coord = primero)

Y1 <- c(0,0,5,5,0)
# Pega X y Y para crear una tabla de coordenadas
c1 <- cbind(X1,Y1)
print(c1)

##      X1 Y1
## [1,]  5  0
## [2,] 10  0
## [3,] 10  5
## [4,]  6  5
## [5,]  5  0

class(c1)

## [1] "matrix"

# Crea el objeto Polygon. Un polygon es una forma simple cerrada
# eventualmente con hueco(s)
P1 <- st_polygon(list(c1))
plot(P1, axes=T)
```

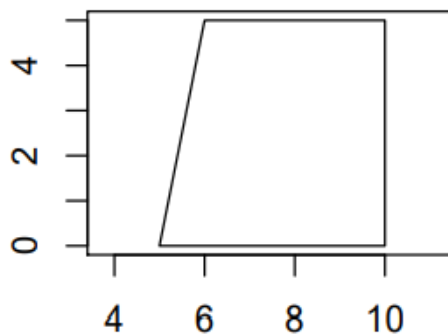


Figura 15. Cobertura de polígonos (Mas, 2018).

2.1.1 Datos raster: Clase RasterLayer en el paquete raster.

El paquete **raster** permite manejar datos raster con el objeto de clase RasterLayer. El raster puede obtenerse a partir de una matriz que contiene los valores de cada celda. La extensión espacial es definida por la función **extent()** introduciendo las coordenadas extremas del raster (x mínimo, x máximo, y mínimo, y máximo).

```
# install.packages("raster") # eliminar comentario para instalar paquete
library(raster)
# Creamos una matriz
m <- matrix(c(1,2,3,4,2,NA,2,2,3,3,3,1),ncol=4,nrow=3,byrow=TRUE)
m

##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    2   NA    2    2
## [3,]    3    3    3    1

r <- raster(m)
extent(r) <- extent(c(0,4,0,3))
class(r)

## [1] "RasterLayer"
## attr(,"package")
## [1] "raster"

print(r)

## class      : RasterLayer
## dimensions  : 3, 4, 12  (nrow, ncol, ncell)
## resolution  : 1, 1  (x, y)
## extent      : 0, 4, 0, 3  (xmin, xmax, ymin, ymax)
## coord. ref. : NA
## data source : in memory
## names       : layer
## values      : 1, 4  (min, max)

plot(r)
```

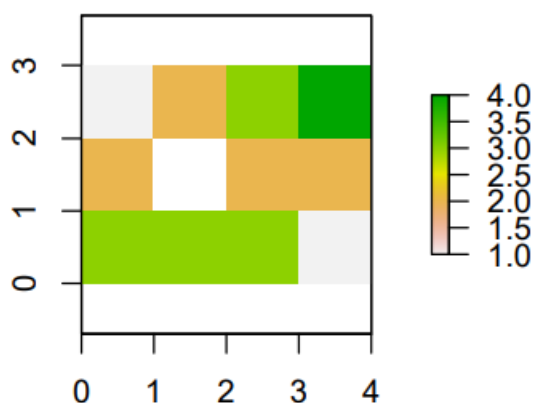


Figura 16. Datos raster (Mas, 2018).

2.2 Construcción de aplicaciones geoespaciales

Los archivos que contienen datos espaciales se conocen comúnmente como **shapefiles**, que generalmente se colocan en una carpeta que contiene al menos tres archivos con las extensiones **.shp**, **.dbf** y **.shx** (Escobar & Ortiz, 2021). En R, los shapefiles se suelen representar mediante vectores, que consisten en la descripción de la geometría o forma (**shape**) de los objetos.

2.2.1 Importación de archivos shape

El formato shapefile es un formato de archivos para datos vectoriales desarrollado por la compañía ESRI y ampliamente utilizado. Está compuesto por lo menos por tres ficheros informáticos que tienen las extensiones siguientes:

- **.shp**: almacena las entidades geométricas de los objetos.
- **.shx**: almacena el índice de las entidades geométricas.
- **.dbf**: almacena la tabla de atributos de los objetos.

Es común encontrar un cuarto archivo con extensión **.prj** que contiene la información referida al sistema de coordenadas. Existen varios paquetes para importar archivos shape en R.

Para trabajar con el paquete **sf**, tenemos que cargar la librería con **library(sf)**. Para poder importar los archivos shape sin indicar la ruta de acceso, declaramos el espacio de trabajo como la carpeta en la cual se encuentran estos archivos. La función **st_read()** permite importar mapas en formato **shape** de cualquier geometría. **class()** nos permite comprobar que el objeto espacial creado es de la clase **sf**, **summary()** nos permite comprobar su estructura de dataframe. Para los objetos de clase **sf**, **plot()** elabora un mapa para cada atributo. Si se desea mapear únicamente la geometría de los objetos (en este caso el límite de los polígonos), se usa la función **st_geometry()**.

En el repositorio <https://www.cursosgis.com/plataformas-para-la-descarga-de-datos-shp-de-america-latina/> puede encontrar algunos archivos **.shp** de América latina.

Ejemplo de carga de archivo .shp.

Primero se deberá instalar el paquete `sf`, y usar la librería `sf`.

```
install.packages("sf")  
library(sf)
```

Para ver la estructura de los objetos de tipo **sf** cargamos nuestra base de datos en el formato **shapefile**, usando la función **read_sf()**. Considerar para este ejemplo, que el archivo *shp_brasil.shp* es tomado como referencia, por tanto, puede ser cualquier archivo, que tengamos a disposición, para la práctica.

```
shp_brasil <- read_sf("datos_espaciales/shp_brasil.shp")  
class(shp_brasil)  
## [1] "sf"          "tbl_df"      "tbl"        "data.frame"
```

Una vez cargado nuestro shapefile, podemos empezar a generar mapas. Ten en cuenta que la forma de hacerlo se basa en **ggplot2**; sólo tenemos que seleccionar nuestra base de datos y añadir el geom **geom_sf()** (Escobar & Ortiz, 2021).

```
ggplot(data = shp_brasil) +  
  geom_sf()
```



Figura 17. Carga de mapa (Escobar & Ortiz, 2021).

2.2.2 Elaboración de mapas.

Existen muchas otras opciones para la elaboración de mapas en R, se pueden mencionar la función **spplot** de **sp** y los paquetes **Choroplethr**, **Prettypmapr** y **tmap**. **RgoogleMaps** y **Rosm** permiten utilizar como fondo los mapas de **GoogleMap**, **Open Street** y **Bing** (Mas, 2018).

Por otro lado, existen numerosos paquetes para acceder a datos geoespaciales como estructuras de datos de características simples. Un par de ejemplos notables son **rnaturalearth** y **USAboundaries**. El paquete **rnaturalearth** es mejor para obtener cualquier dato cartográfico del mundo a través de una API proporcionada por <https://www.naturalearthdata.com/> (Sur 2017). El paquete **USAboundaries** es estupendo para obtener datos cartográficos de los Estados. Realmente no importa qué herramienta se utilice para obtener o crear un objeto **sf**.

```

library(rnaturalearth)
world <- ne_countries(returnclass = "sf")
class(world)
#> [1] "sf"      "data.frame"
plot_ly(world, color = I("gray90"), stroke = I("black"), span = I(1))

```



Figura 18. Uso de mapa disponible en paquete **rnaturalearth**.

2.2.2 Elaboración de cartogramas.

Los cartogramas distorsionan el tamaño de los polígonos geoespaciales para codificar una variable numérica distinta del tamaño del terreno. Existen numerosos tipos de cartogramas y se suelen clasificar por su capacidad para conservar la forma y mantener las regiones contiguas. Se ha demostrado que los cartogramas son un enfoque eficaz tanto para codificar como para enseñar sobre datos geoespaciales, aunque los efectos ciertamente varían según el tipo de cartograma. El paquete R **cartogram** ofrece una interfaz para varios algoritmos populares de cartogramas. Varios otros paquetes de R proporcionan algoritmos de cartograma, pero lo mejor de cartograma es que todas las funciones pueden tomar un objeto **sf** (o **sp**) como entrada y devolver un objeto **sf**. Esto hace que sea increíblemente fácil pasar de objetos espaciales sin procesar a objetos transformados y a visuales (Sievert, 2019).

La figura 19, muestra un cartograma de área continua de la población de EE.UU., en 2014 utilizando un algoritmo de distorsión de hoja de goma.

```
library(cartogram)
library(albersusa)

us_cont <- cartogram_cont(usa_sf("laea"), "pop_2014")

plot_ly(us_cont) %>%
  add_sf(
    color = ~pop_2014,
    split = ~name,
    span = I(1),
    text = ~paste(name, scales::number_si(pop_2014)),
    hoverinfo = "text",
    hoveron = "fills"
  ) %>%
  layout(showlegend = FALSE) %>%
  colorbar(title = "Population \n 2014")
```

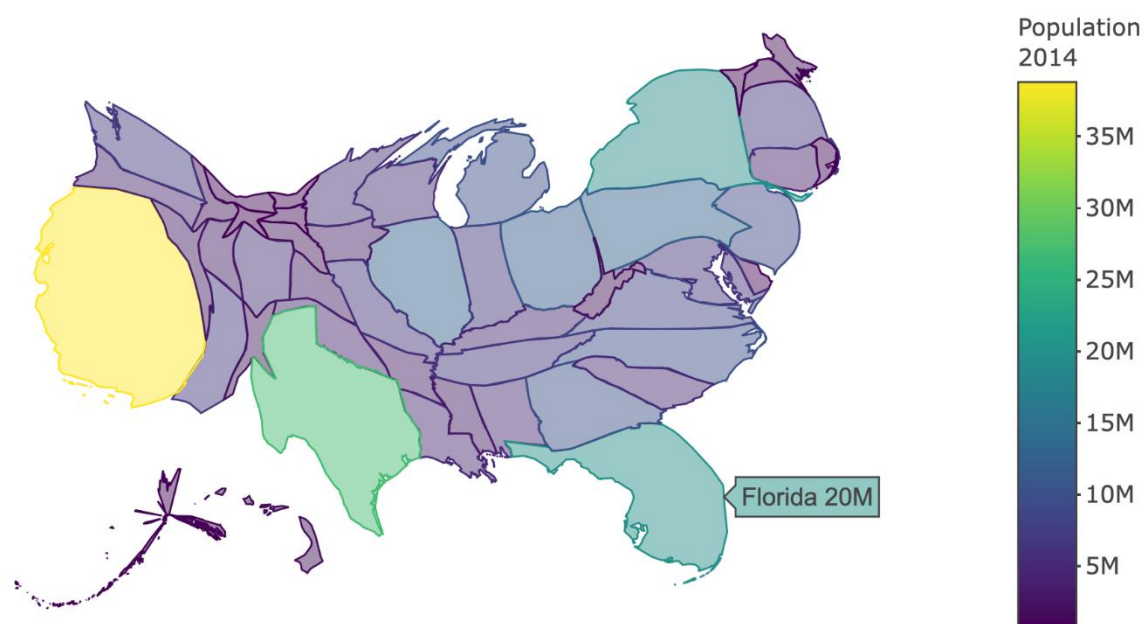


Figura 19. Elaboración de cartograma.

EJEMPLOS PARA PRÁCTICAS Y EXPERIMENTACIÓN

1. Creación de mapas básicos con map() del paquete maps

```
require(maps)  
map()
```



```
require(maps)
map(col = "cornflowerblue")
```



```
require(maps)
if(require(mapproj)) { # mapproj is used for projection="polyconic"
  # color US county map by 2009 unemployment rate
  # match counties to map using FIPS county codes
  # Based on J's solution to the "Choropleth Challenge"
  # Code improvements by Hack-R (hack-r.github.io)
  # load data
  # unemp includes data for some counties not on the "lower 48 states" county
  # map, such as those in Alaska, Hawaii, Puerto Rico, and some tiny Virginia
  # cities
  data(unemp)
  data(county.fips)

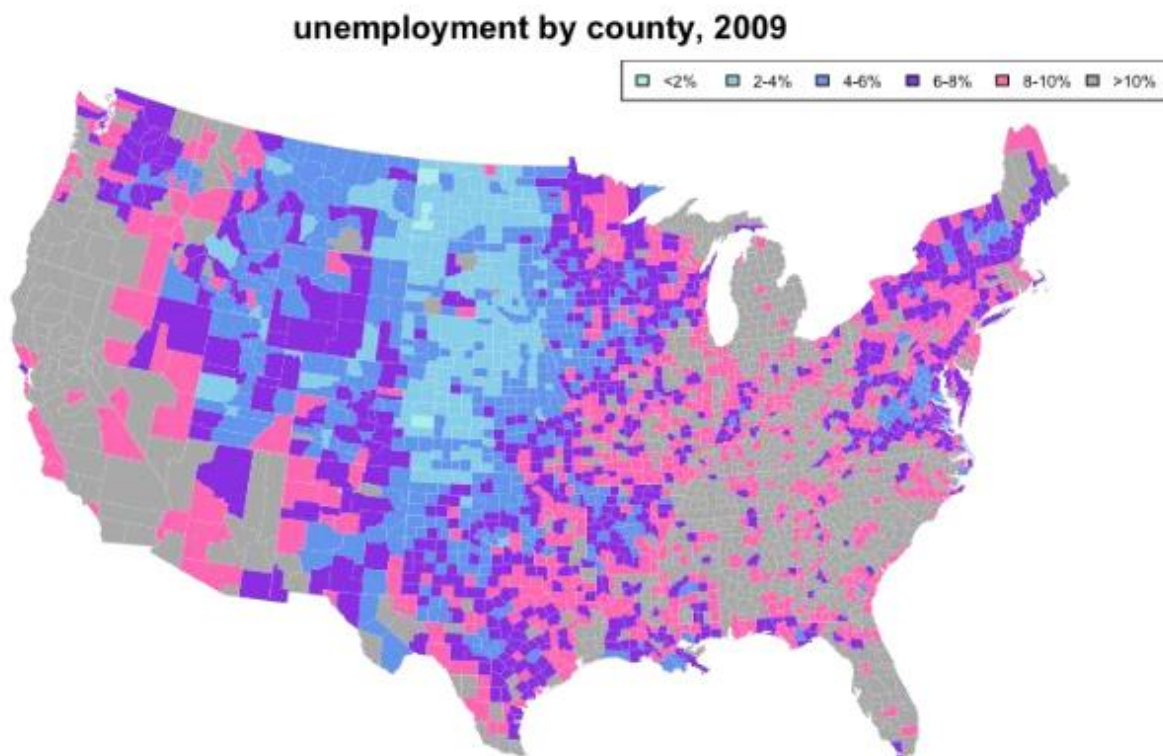
  # define color buckets
  colors = c("paleturquoise", "skyblue", "cornflowerblue", "blueviolet", "hotpink",
"darkgrey")
  unemp$colorBuckets <- as.numeric(cut(unemp$unemp, c(0, 2, 4, 6, 8, 10, 100)))
GoalKicker.com - R Notes for Professionals 163
  leg.txt <- c("<2%", "2-4%", "4-6%", "6-8%", "8-10%", ">10%")

  # align data with map definitions by (partial) matching state,county
  # names, which include multiple polygons for some counties
  cnty.fips <- county.fips$fips[match(map("county", plot=FALSE)$names,
                                         county.fips$polynome)]
  colorsmatched <- unemp$colorBuckets[match(cnty.fips, unemp$fips)]

  # draw map
```



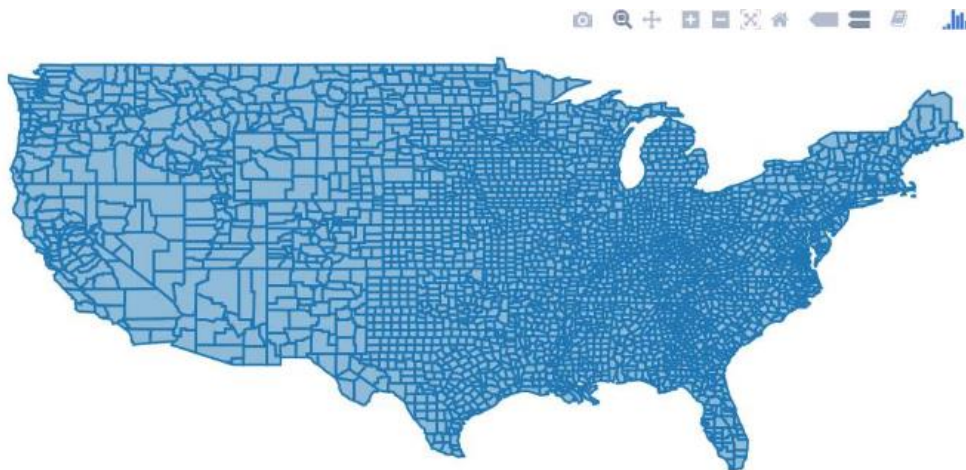
```
par(mar=c(1, 1, 2, 1) + 0.1)
map("county", col = colors[colorsmatched], fill = TRUE, resolution = 0,
    lty = 0, projection = "polyconic")
map("state", col = "white", fill = FALSE, add = TRUE, lty = 1, lwd = 0.1,
    projection="polyconic")
title("unemployment by county, 2009")
legend("topright", leg.txt, horiz = TRUE, fill = colors, cex=0.6)
}
```



2. Mapas interactivos plotly

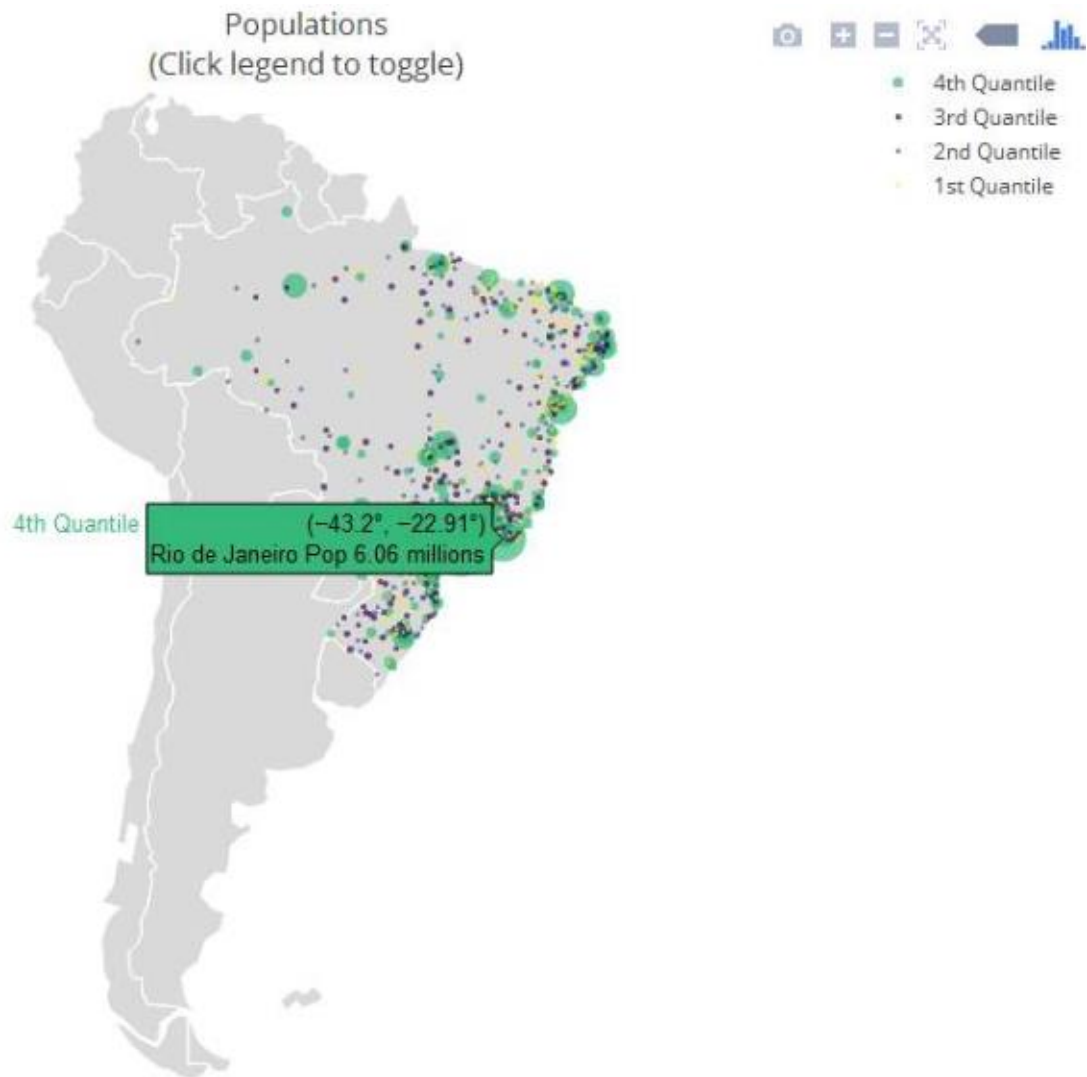
El paquete **plotly** permite realizar muchos tipos de gráficos interactivos, incluidos los mapas. Hay varias maneras de crear un mapa en plotly. Puede suministrar los datos del mapa usted mismo (a través de `plot_ly()` o `ggplotly()`), utilizar las capacidades de mapeo "nativo" de plotly de plotly (mediante `plot_geo()` o `plot_mapbox()`), o incluso una combinación de ambas. Un ejemplo de suministro del mapa sería:

```
library(plotly)
map_data("county") %>%
  group_by(group) %>%
  plot_ly(x = ~long, y = ~lat) %>%
  add_polygons() %>%
  layout(
    xaxis = list(title = "", showgrid = FALSE, showticklabels = FALSE),
    yaxis = list(title = "", showgrid = FALSE, showticklabels = FALSE)
  )
```



Para una combinación de ambos enfoques, cambie `plot_ly()` por `plot_geo()` o `plot_mapbox()` en el ejemplo anterior. El siguiente ejemplo es un enfoque "estrictamente nativo" que aprovecha el atributo `layout.geo` para establecer la estética y el nivel de zoom del mapa. También utiliza la base de datos `world.cities` de `maps` para filtrar las ciudades brasileñas y trazarlas sobre el mapa "nativo". Las principales variables: **pophis** un texto con la ciudad y su población (que se muestra al pasar el ratón por encima); **qes** un factor ordenado del cuantil de la población. `ge` tiene información para el diseño de los mapas. Consulte la documentación del paquete para más información.

```
library(maps)
dfb <- world.cities[world.cities$country.etc=="Brazil",]
library(plotly)
dfb$poph <- paste(dfb$name, "Pop", round(dfb$pop/1e6,2), " millions")
dfb$q <- with(dfb, cut(pop, quantile(pop), include.lowest = T))
levels(dfb$q) <- paste(c("1st", "2nd", "3rd", "4th"), "Quantile")
dfb$q <- as.ordered(dfb$q)
ge <- list(
  scope = 'south america',
  showland = TRUE,
  landcolor = toRGB("gray85"),
  subunitwidth = 1,
  countrywidth = 1,
  subunitcolor = toRGB("white"),
  countrycolor = toRGB("white")
)
plot_geo(dfb, lon = ~long, lat = ~lat, text = ~poph,
  marker = ~list(size = sqrt(pop/10000) + 1, line = list(width = 0)),
  color = ~q, locationmode = 'country names') %>%
  layout(geo = ge, title = 'Populations<br>(Click legend to toggle)')
```

Recuerde que: Los archivos que contienen datos espaciales se conocen comúnmente como shapefiles.

Bibliografía:

Amat, J. (2017). Text mining con R: ejemplo práctico Twitter. *RPubs*.

https://rpubs.com/Joaquin_AR/334526

Botta-Ferret, E., & Cabrera-Gato, J. E. (2007). Minería de textos: Una herramienta útil para mejorar la gestión del bibliotecario en el entorno digital. *ACIMED*, 16(4).

https://www.researchgate.net/publication/28184496_Mineria_de_textos_una_herramienta_util_para_mejorar_la_gestion_del_bibliotecario_en_el_entorno_digital

- Chang, J., Reilly, C., Pontika, N., Owen, G., Haug, K., & Oudenhoven, M. (2018, febrero). *¿Qué es la minería de textos, cómo funciona y por qué es útil?* | . Blog: Universo Abierto. <https://universoabierto.org/2018/02/22/que-es-la-mineria-de-textos-como-funciona-y-por-que-es-util/>
- Contreras Barrera, M. (2014). *Minería de texto: una visión actual*.
<https://www.redalyc.org/pdf/285/28540279005.pdf>
- DataScientest. (2022, marzo). *Text Mining o minería de textos: definición, técnicas, casos de uso*. <https://datascientest.com/es/text-mining-o-mineria-de-textos-definicion-tecnicas-casos-de-uso>
- Escobar, A., & Ortiz, G. (2021). *Mapas y datos espaciales*.
<https://arcruz0.github.io/libroadp/maps.html>
- Fernández Casal, R., & Cotos Yáñez, T. (2021). *Estadística Espacial con R*.
https://rubenfcasal.github.io/estadistica_espacial/estadistica_espacial.pdf
- Guerrero Restrepo, J. M. (2020). *Aplicación de técnicas de minería de texto para el descubrimiento de relaciones conceptuales entre los trabajos de grado de la Universidad de Nariño*.
http://datamining.dc.uba.ar/datamining/Files/Tesis/Tesis_Jimmy_Guerrero.pdf
- Mas, J.-F. (2018). *Análisis espacial con R. Usar R como sistema de información geográfica*. <http://eujournal.org/files/journals/1/books/JeanFrancoisMas.pdf>
- Sievert, C. (2019). *Interactive web-based data visualization with R, plotly, and shiny* .
<https://plotly-r.com/maps.html#maps-custom>
- Silge, J., & Robinson, D. (2016). *Text Mining with R - Preface* (4.2.0 (202)).
<https://www.tidytextmining.com/index.html>
- UMA. (2022, mayo). *¿Qué es el Text Mining y cuáles son sus aplicaciones?* - Máster en Big Data e Inteligencia Artificial Málaga. Universidad de Málaga.
<https://www.bigdata.uma.es/que-es-el-text-mining-y-cuales-son-sus-aplicaciones/>